

Um Microkernel Simples para RISC-V

Implementação de um Sistema Operacional Minimalista para disciplina de SO

Felipe Viel

23 de junho de 2026

Sumário

1	Introdução	9
2	Ambiente de Desenvolvimento	10
2.1	Instalação no GitHub Codespaces	10
2.2	Verificação	10
3	Estrutura do Projeto	11
4	Sistema de Build: Makefile e linker.ld	12
4.1	Visão Geral do Processo de Build	12
4.2	Análise do Makefile	12
4.2.1	Código Completo	12
4.3	Variável CROSS	13
4.4	Compilador e Linker	13
4.5	Flags de Compilação	14
4.5.1	-march=rv64gc	14
4.5.2	-mabi=lp64	14
4.5.3	-mmodel=medany	14
4.5.4	-ffreestanding	14
4.5.5	-nostdlib	14
4.5.6	-nostartfiles	14
4.5.7	-fno-stack-protector	14
4.6	Lista de Objetos	15
4.7	Regra all	15
4.8	Regra kernel.elf	15
4.9	Regras de Compilação	15
4.9.1	Assembly	15
4.9.2	C	15
4.9.3	Variável \$<	15
4.10	Regra clean	15
4.11	Regra run	16
4.12	Análise do Linker Script	16
4.12.1	Código Completo	16
4.13	ENTRY(_start)	16
4.14	Endereço Base	16
4.15	Seção .text	17
4.16	Seção .rodata	17
4.17	Seção .data	17
4.18	Seção .bss	17
4.19	ALIGN(4096)	17
4.20	__global_pointer\$	17

4.21	Por que o Linker Script é Necessário?	17
4.22	Ferramentas de Inspeção	18
4.22.1	readelf	18
4.22.2	objdump	18
4.22.3	nm	18
4.23	Resumo Final	18
5	Boot do Sistema	19
5.1	Código Completo do <code>start.S</code>	19
5.2	Visão Geral do Processo de Boot	20
5.3	Diretiva <code>.section .text.boot</code>	20
5.4	Diretiva <code>.global _start</code>	20
5.5	Rótulo <code>_start</code>	20
5.6	Inicialização do Registrador <code>gp</code>	20
5.6.1	O que é o Registrador <code>gp</code> ?	20
5.6.2	Definição no Linker Script	21
5.6.3	Por que Isso é Importante?	21
5.7	Inicialização da Stack	21
5.7.1	O que é o Registrador <code>sp</code> ?	21
5.7.2	Por que Usar o Topo da Stack?	21
5.8	Reserva da Stack	21
5.8.1	<code>.space 4096</code>	21
5.8.2	<code>kernel_stack_top</code>	21
5.9	Alinhamento com <code>.align 12</code>	21
5.10	Zeragem da Seção <code>.bss</code>	22
5.10.1	Motivação	22
5.10.2	Símbolos Fornecidos pelo Linker	22
5.10.3	Código	22
5.10.4	Explicação	22
5.11	Rótulos Locais Numéricos	22
5.12	Chamada de <code>kernel_main()</code>	22
5.13	Loop Infinito de Segurança	22
5.14	Registradores Utilizados	23
5.15	Fluxo de Execução	23
5.16	Por que <code>start.S</code> é Essencial?	23
5.17	Resumo Final	23
6	Driver UART	24
6.1	Arquivos do Módulo	24
6.2	Arquivo <code>uart.h</code>	24
6.2.1	Código Completo	24
6.3	Objetivo do Header	24
6.4	Diretivas de Inclusão	24
6.4.1	Include Guard	24
6.5	Arquivo <code>uart.c</code>	25
6.5.1	Código Completo	25
6.6	Mapeamento de Memória	25
6.7	Definição da Constante	25
6.7.1	Por que usar <code>L</code> ?	25
6.8	Função <code>uart_putc()</code>	26
6.8.1	Código	26
6.8.2	Objetivo	26

6.8.3	Conversão de Endereço em Ponteiro	26
6.8.4	Uso de <code>volatile</code>	26
6.8.5	Desreferenciação	26
6.8.6	Escrita do Byte	26
6.9	Exemplo Prático	26
6.10	Função <code>uart_print()</code>	27
6.10.1	Código	27
6.10.2	Objetivo	27
6.11	Strings em C	27
6.12	Funcionamento do Loop	27
6.13	Expressão <code>*s++</code>	27
6.14	Exemplo de Execução	27
6.15	Integração com o QEMU	28
6.16	Exemplo no Kernel	28
6.17	Por que Não Usar <code>printf()</code> ?	28
6.18	Vantagens da Implementação	28
6.19	Limitações	28
6.20	Fluxo Completo de Impressão	29
6.21	Resumo Final	29
7	Gerência de Memória	30
7.1	Objetivo do Módulo	30
7.2	Arquivos do Módulo	30
7.3	Arquivo <code>memory.h</code>	30
7.3.1	Código Completo	30
7.4	Funções Declaradas	30
7.5	Arquivo <code>memory.c</code>	31
7.5.1	Código Completo	31
7.6	Conceito de Heap	31
7.7	Definição do Heap	31
7.8	Ponteiro de Alocação	32
7.9	Visualização Inicial	32
7.10	Função <code>memory_init()</code>	32
7.11	Quando é Chamada?	32
7.12	Função <code>kmalloc()</code>	32
7.13	Passo 1: Alinhamento para 8 Bytes	32
7.13.1	Motivação	33
7.13.2	Exemplos	33
7.13.3	Explicação Bit a Bit	33
7.14	Passo 2: Verificação de Espaço	33
7.15	Passo 3: Definição do Ponteiro	33
7.16	Passo 4: Avanço do Heap	33
7.17	Passo 5: Retorno	33
7.18	Função <code>memory_used()</code>	33
7.19	Exemplo Prático	34
7.20	Uso na Criação de Tasks	34
7.21	Representação Visual	34
7.22	Características do Bump Allocator	34
7.23	Limitações	34
7.24	Complexidade	34
7.25	Por que Essa Implementação é Adequada?	35
7.26	Resumo Final	35

8	Gerenciamento de Tasks: Análise Detalhada de <code>task.h</code> e <code>task.c</code>	36
8.1	Conceito de Task	36
8.2	Arquivos do Módulo	36
8.3	Arquivo <code>task.h</code>	37
8.3.1	Código Completo	37
8.4	Constante <code>MAX_TASKS</code>	37
8.5	Estrutura TCB	37
8.5.1	<code>regs[31]</code>	37
8.5.2	<code>sepc</code>	38
8.5.3	<code>entry</code>	38
8.5.4	<code>priority</code>	38
8.5.5	<code>state</code>	38
8.5.6	<code>stack</code>	39
8.5.7	<code>stack_size</code>	39
8.6	Variáveis Globais	39
8.7	Função <code>xTaskCreate()</code>	39
8.8	Arquivo <code>task.c</code>	39
8.8.1	Código Completo	39
8.9	Definição das Variáveis Globais	40
8.10	Validação do Número Máximo	40
8.11	Seleção do Próximo TCB	40
8.12	Inicialização de Campos	40
8.13	Alocação da Stack	41
8.14	Verificação de Falha	41
8.15	Inicialização do Stack Pointer	41
8.16	Preparação do Contexto Inicial	41
8.16.1	Registrador <code>ra</code>	41
8.16.2	Registrador <code>sp</code>	41
8.16.3	Registrador <code>sepc</code>	41
8.17	Por que Salvar <code>ra</code> e <code>sepc</code> ?	41
8.18	Fluxo de Criação de Task	42
8.19	Exemplo de Uso	42
8.20	Resumo Final	42
9	Chaveamento de Contexto	43
9.1	Objetivo do Chaveamento de Contexto	43
9.2	Registradores RISC-V: Nome Arquitetural vs Nome ABI	43
9.3	Estrutura do Contexto no TCB	44
9.4	Assinatura da Função	44
9.5	Código em Assembly	45
9.6	Parâmetros em RISC-V	45
9.7	Etapa 1: Salvamento do Contexto Atual	45
9.8	Etapa 2: Restauração do Próximo Contexto	46
9.9	Etapa 3: Execução de <code>ret</code>	46
9.10	Inicialização do Contexto em <code>xTaskCreate()</code>	46
9.11	Exemplo Prático	46
9.12	Por que Salvar Todos os Registradores?	46
9.13	Separação de Responsabilidades	47
9.14	Fluxograma do Context Switch	47
9.15	Resumo	47
10	Escalonador	48

10.1	Arquivos do Módulo	48
10.2	Objetivos do Módulo	48
10.3	Arquivo <code>scheduler.h</code>	48
10.3.1	Código Completo	48
10.4	Tipo <code>sched_algo_t</code>	49
10.5	Funções Públicas	49
10.6	Arquivo <code>scheduler.c</code>	49
10.6.1	Código Completo	49
10.7	Variáveis Globais	51
10.7.1	<code>current</code>	51
10.7.2	<code>current_algo</code>	51
10.8	Algoritmo Round-Robin	51
10.9	Função <code>scheduler_set_algorithm()</code>	51
10.10	Função <code>scheduler_current_task()</code>	52
10.11	Função <code>yield()</code>	52
10.11.1	Objetivo	52
10.11.2	Código	52
10.11.3	Fluxo	52
10.12	Função <code>schedule_from_trap()</code>	52
10.12.1	Objetivo	52
10.12.2	Parâmetro	53
10.12.3	Etapas	53
10.12.4	Por que não usar <code>context_switch()</code> ?	53
10.13	Função <code>scheduler_start()</code>	53
10.14	Fluxo Cooperativo	53
10.15	Fluxo Preemptivo	54
10.16	Separação de Responsabilidades	54
10.17	Resumo Final	54
11	Timer	55
11.1	Por que o Timer Ainda Não Foi Implementado?	55
11.2	Escalonamento Cooperativo vs. Preemptivo	55
11.3	Como Seria a Evolução	55
11.4	Resumo	56
12	Traps e Interrupções	57
12.1	O que é um Trap?	57
12.2	Classificação Oficial no RISC-V	57
12.2.1	Interrupções (Interrupts)	57
12.2.2	Exceções (Exceptions)	57
12.3	Motivação	58
12.4	O que é um Trap?	58
12.5	Quando o Suporte a Traps Será Necessário?	58
12.6	Funcionamento Atual do Escalonamento	58
12.7	Possível Evolução	59
12.8	Resumo	59
13	Compilação e Execução	60
13.1	Compilação	60
13.2	Execução	60
13.3	Finalizar Qemu no Terminal	60
13.4	Saída Esperada	60

14 Tutorial de Implementação do Escalonamento Preemptivo	61
14.1 Objetivo Final	61
14.2 Arquivos Necessários	61
14.2.1 Novos Arquivos	61
14.2.2 Arquivos a Modificar	62
14.3 Etapa 1: Implementação do Timer	62
14.3.1 Arquivo <code>timer.h</code>	62
14.3.2 Função <code>timer_init()</code>	62
14.3.3 Função <code>timer_next()</code>	62
14.3.4 Integração com OpenSBI	62
14.4 Etapa 2: Implementação do Trap Handler	62
14.4.1 Arquivo <code>trap.h</code>	62
14.4.2 Parâmetro <code>frame</code>	63
14.4.3 Função <code>trap_handler()</code>	63
14.5 Etapa 3: Implementação de <code>trap_entry.S</code>	63
14.5.1 Responsabilidades	63
14.6 Etapa 4: Extensão do Scheduler	63
14.6.1 Arquivo <code>scheduler.h</code>	63
14.6.2 Função <code>schedule_from_trap()</code>	63
14.7 Etapa 5: Reutilização do Round-Robin	64
14.8 Etapa 6: Configuração do <code>main.c</code>	64
14.8.1 Declaração Necessária	64
14.8.2 Configuração do Vetor de Trap	64
14.8.3 Inicialização do Timer	64
14.9 Etapa 7: Atualização do Makefile	64
14.10 Funcionamento do Round-Robin	65
14.11 Fluxo Completo de Execução	65
14.12 Teste do Sistema	65
14.13 Checklist Final	65
14.13.1 Timer	65
14.13.2 Trap	65
14.13.3 Scheduler	66
14.13.4 Main	66
14.13.5 Build	66
14.14 Resultado Esperado	66
15 Tutorial Guiado: Fluxo de Implementação do Timer, Trap e Escalonamento Round-Robin	67
15.1 Objetivos	67
15.2 Arquivos Envolvidos	67
15.3 Passo 1: Interface do Timer	68
15.3.1 <code>timer.h</code>	68
15.3.2 Objetivo	68
15.4 Passo 2: Implementação do Timer	68
15.4.1 <code>timer.c</code>	68
15.4.2 Desafio do Aluno	69
15.5 Passo 3: Interface do Trap Handler	69
15.5.1 <code>trap.h</code>	69
15.6 Passo 4: Implementação do Trap Handler	69
15.6.1 <code>trap.c</code>	69
15.6.2 Desafio do Aluno	70
15.7 Passo 5: Ponto de Entrada do Trap	70

15.7.1	<code>trap_entry.S</code>	70
15.7.2	Desafio do Aluno	70
15.8	Passo 6: Extensão da Interface do Scheduler	70
15.8.1	<code>scheduler.h</code>	70
15.9	Passo 7: Implementação do Round-Robin	71
15.9.1	Função Básica	71
15.10	Passo 8: Escalonamento a Partir do Trap	71
15.10.1	Trecho de <code>scheduler.c</code>	71
15.10.2	Desafio do Aluno	71
15.11	Passo 9: Configuração no <code>main.c</code>	72
15.12	Passo 10: Teste	72
15.13	Fluxo Completo	73
15.14	Checklist	73
15.15	Resultado Esperado	73
16	Proposta de Evolução do Sistema de Gerência de Memória	74
16.1	Introdução	74
16.2	Limitações do Allocator Atual	74
16.2.1	Exemplo	75
16.3	Problemas do Modelo Linear	75
16.3.1	Ausência de <code>kfree()</code>	75
16.3.2	Exaustão Progressiva do Heap	76
16.3.3	Ausência de Reutilização	76
16.3.4	Pouco Realismo	76
16.4	Proposta: Free List Allocator	76
16.5	Estrutura Conceitual do Heap	76
16.6	Estrutura do Cabeçalho	76
16.6.1	Arquivo <code>memory.c</code>	76
16.6.2	Campo <code>size</code>	77
16.6.3	Campo <code>free</code>	77
16.6.4	Campo <code>next</code>	77
16.7	Estrutura Inicial do Heap	77
16.8	Estratégia de Busca: First Fit	78
16.9	Por que First Fit?	78
16.10	Comparação com Outras Estratégias	78
16.11	Novo Arquivo <code>memory.h</code>	78
16.12	Nova Organização do Heap	78
16.12.1	Heap	79
16.12.2	Lista Livre	79
16.13	Inicialização do Heap	79
16.13.1	Esqueleto	79
16.13.2	Funcionamento	79
16.14	Fluxo do <code>kmalloc()</code>	79
16.15	Esqueleto do <code>kmalloc()</code>	80
16.16	Divisão de Blocos	80
16.17	Fluxo do <code>kfree()</code>	81
16.18	Esqueleto do <code>kfree()</code>	81
16.19	Coalescência	81
16.20	Fragmentação	81
16.20.1	Fragmentação Interna	81
16.20.2	Fragmentação Externa	81
16.21	Esqueleto do <code>memory_used()</code>	82

16.22Esqueleto do <code>memory_free()</code>	82
16.23Integração com Tasks	82
16.24Atividades Propostas	83
16.25Pesquisas Recomendadas	83
16.26Saída Esperada	83
16.27Objetivos de Aprendizagem	83
16.28Conclusão	83
17 Implementação de um Sistema de Arquivos Hierárquico Baseado em Inodes (TreeFS)	84
17.1 Introdução	84
17.2 Objetivos de Aprendizagem	85
17.3 Sistemas de Arquivos Hierárquicos	85
17.4 Por Que Sistemas Hierárquicos Existem?	86
17.5 Entendendo Estruturas Hierárquicas	86
17.6 A Analogia com uma Árvore Genealógica	87
17.7 Metadados e Dados	88
17.8 O Que é um Inode?	88
17.9 A Analogia com uma Biblioteca	88
17.10Inodes em Detalhes	89
17.11Campos do Inode	89
17.11.1 Campo Size	89
17.11.2 Campo Type	90
17.11.3 Campo Links	90
17.11.4 Campo Blocks	91
17.12Arquitetura Geral do TreeFS	91
17.13Organização Física do Disco	91
17.14Superblock	92
17.15Gerenciamento de Inodes Livres	92
17.16Bitmap de Inodes	93
17.17Bitmap de Blocos	93
17.18Primeira Estrutura do Sistema	93
17.19Inicialização do Sistema	93
17.20Alocação de Inodes	94
17.21Liberação de Inodes	94
17.22Diretórios	95
17.23Entradas de Diretório	95
17.24Resolução de Caminhos	96
17.25Implementação de <code>path_lookup()</code>	96
17.26Navegação na Árvore	97
17.27Criação de Diretórios	97
17.28Implementação de <code>mkdir()</code>	98
17.29Criação de Arquivos	98
17.30Implementação de <code>create()</code>	99
17.31Escrita de Arquivos	99
17.32Implementação de <code>write()</code>	99
17.33Leitura de Arquivos	100
17.34Implementação de <code>read()</code>	100
17.35Remoção de Arquivos	100
17.36Implementação de <code>unlink()</code>	101
17.37Validação da Implementação	101
17.38Conclusão	102

Capítulo 1

Introdução

Este projeto apresenta a implementação de um microkernel educacional para a arquitetura RISC-V (RV64), executado em modo supervisor (S-mode) e simulado no QEMU utilizando OpenSBI.

Os principais componentes implementados são:

- Boot em Assembly;
- UART para saída serial;
- Gerência de memória com `kmalloc()`;
- Criação de tasks no estilo FreeRTOS;
- Chaveamento de contexto completo;
- Escalonador Round-Robin;
- Preempção por timer;
- Tratamento de traps e interrupções.

Capítulo 2

Ambiente de Desenvolvimento

2.1 Instalação no GitHub Codespaces

```
sudo apt update
sudo apt install -y \
    gcc-riscv64-unknown-elf \
    binutils-riscv64-unknown-elf \
    qemu-system-misc \
    make \
    gdb-multiarch
```

2.2 Verificação

```
riscv64-unknown-elf-gcc --version
qemu-system-riscv64 --version
```

Capítulo 3

Estrutura do Projeto

Arquivo	Função
boot/start.S	Boot inicial e runtime mínimo
boot/trap_entry.S	Entrada de traps
kernel/main.c	Inicialização do kernel
kernel/uart.c	Saída serial
kernel/memory.c	Heap do kernel
kernel/task.c	Criação de tasks
kernel/context.S	Context switch
kernel/scheduler.c	Escalonador
kernel/timer.c	Timer via SBI
kernel/trap.c	Tratamento de interrupções
linker.ld	Layout da memória
Makefile	Compilação

Capítulo 4

Sistema de Build: Makefile e linker.ld

O processo de geração do microkernel depende de dois arquivos fundamentais:

- **Makefile**: automatiza a compilação e a linkedição.
- **linker.ld**: define o layout do kernel na memória.

Esses arquivos determinam, respectivamente:

- **como** o kernel será construído;
- **onde** cada seção do executável será posicionada na memória.

4.1 Visão Geral do Processo de Build

O fluxo completo é:

1. Compilar cada arquivo `.c` e `.S` em arquivos objeto `.o`.
2. Unir todos os objetos em um único executável ELF.
3. Posicionar as seções do ELF conforme o script de linkedição.
4. Executar o ELF no QEMU.

```
start.S
main.c
scheduler.c
task.c
↓ compilação
*.o
↓ linkedição com linker.ld
kernel.elf
↓ OpenSBI + QEMU
Microkernel em execução
```

4.2 Análise do Makefile

4.2.1 Código Completo

```
CROSS = riscv64-unknown-elf-

CC = $(CROSS)gcc
LD = $(CROSS)ld
```

```

CFLAGS = -march=rv64gc -mabi=lp64 \
        -mcmodel=medany \
        -ffreestanding \
        -nostdlib \
        -nostartfiles \
        -fno-stack-protector \
        -Wall

OBJS = \
    start.o \
    context.o \
    main.o \
    uart.o \
    memory.o \
    task.o \
    scheduler.o

all: kernel.elf

kernel.elf: $(OBJS)
    $(LD) -T linker.ld $(OBJS) -o kernel.elf

%.o: boot/%.S
    $(CC) $(CFLAGS) -c $<

%.o: kernel/%.c
    $(CC) $(CFLAGS) -Iinclude -c $<

clean:
    rm -f *.o kernel.elf

run: kernel.elf
    qemu-system-riscv64 \
        -machine virt \
        -m 128M \
        -nographic \
        -bios default \
        -kernel kernel.elf

```

4.3 Variável CROSS

```
CROSS = riscv64-unknown-elf-
```

Define o prefixo do compilador cruzado.

Ferramentas utilizadas:

- riscv64-unknown-elf-gcc
- riscv64-unknown-elf-ld
- riscv64-unknown-elf-objdump
- riscv64-unknown-elf-readelf

4.4 Compilador e Linker

```
CC = $(CROSS)gcc
LD = $(CROSS)ld
```

- CC: compilador.
- LD: linker.

4.5 Flags de Compilação

```
CFLAGS = -march=rv64gc -mabi=lp64 \
        -mcmodel=medany \
        -ffreestanding \
        -nostdlib \
        -nostartfiles \
        -fno-stack-protector \
        -Wall
```

4.5.1 -march=rv64gc

Gera código para RISC-V de 64 bits com as extensões:

- I: conjunto base.
- M: multiplicação/divisão.
- A: operações atômicas.
- F/D: ponto flutuante.
- C: instruções comprimidas.

4.5.2 -mabi=lp64

Define:

- `long` = 64 bits;
- ponteiros = 64 bits.

4.5.3 -mcmodel=medany

Permite acesso a símbolos em posições arbitrárias, como `0x80200000`.

4.5.4 -ffreestanding

Indica que não existe sistema operacional hospedeiro.

4.5.5 -nostdlib

Impede a inclusão automática da biblioteca padrão C.

4.5.6 -nostartfiles

Impede inclusão de arquivos de inicialização padrão.

4.5.7 -fno-stack-protector

Desabilita verificações de proteção de stack que exigiriam suporte adicional.

4.6 Lista de Objetos

```
OBJS = ...
```

Lista todos os arquivos objeto que compõem o kernel.

Objeto	Função
start.o	Boot inicial
context.o	Troca de contexto
main.o	kernel_main()
uart.o	Saída serial
memory.o	Alocador simples
task.o	Criação de tasks
scheduler.o	Escalonador

4.7 Regra all

```
all: kernel.elf
```

Meta padrão executada quando o usuário digita:

```
make
```

4.8 Regra kernel.elf

```
kernel.elf: $(OBJS)
    $(LD) -T linker.ld $(OBJS) -o kernel.elf
```

Une todos os arquivos objeto em um único executável ELF.

4.9 Regras de Compilação

4.9.1 Assembly

```
%.o: boot/%.S
    $(CC) $(CFLAGS) -c $<
```

4.9.2 C

```
%.o: kernel/%.c
    $(CC) $(CFLAGS) -Iinclude -c $<
```

4.9.3 Variável \$<

Representa o primeiro pré-requisito da regra.

4.10 Regra clean

```
clean:
    rm -f *.o kernel.elf
```

Remove arquivos gerados.

4.11 Regra run

```
run: kernel.elf
    qemu-system-riscv64 ...
```

Executa automaticamente o kernel no QEMU.

4.12 Análise do Linker Script

4.12.1 Código Completo

```
ENTRY(_start)

SECTIONS
{
    . = 0x80200000;

    .text : {
        *(.text.boot)
        *(.text*)
    }

    .rodata : {
        *(.rodata*)
    }

    .data : {
        *(.data*)
    }

    .bss : ALIGN(4096) {
        __bss_start = .;
        *(.bss*)
        *(COMMON)
        __bss_end = .;
    }

    __global_pointer$ = . + 0x800;
}
```

4.13 ENTRY(_start)

Define o símbolo inicial do executável.

4.14 Endereço Base

```
. = 0x80200000;
```

Corresponde ao endereço informado pelo OpenSBI como:

```
Domain0 Next Address : 0x0000000080200000
```

4.15 Seção .text

Contém código executável.

```
.text : {  
    *.text.boot  
    *.text*  
}
```

O código de boot é posicionado primeiro.

4.16 Seção .rodata

Contém constantes somente leitura, como strings.

4.17 Seção .data

Contém variáveis globais inicializadas.

4.18 Seção .bss

Contém variáveis globais não inicializadas.

```
__bss_start = .;  
...  
__bss_end = .;
```

Esses símbolos são usados em `start.S` para zerar a seção.

4.19 ALIGN(4096)

Alinha a `.bss` em uma fronteira de página.

4.20 __global_pointer\$

```
__global_pointer$ = . + 0x800;
```

Define o valor inicial do registrador `gp`.

4.21 Por que o Linker Script é Necessário?

Em kernels bare-metal, o programador deve definir explicitamente:

- endereço de carregamento;
- ordem das seções;
- símbolos especiais;
- ponto de entrada.

4.22 Ferramentas de Inspeção

4.22.1 readelf

```
riscv64-unknown-elf-readelf -h kernel.elf
```

4.22.2 objdump

```
riscv64-unknown-elf-objdump -h kernel.elf
```

4.22.3 nm

```
riscv64-unknown-elf-nm kernel.elf
```

4.23 Resumo Final

- O Makefile automatiza a construção do kernel.
- O `linker.ld` define a organização do kernel na memória.
- O endereço base é `0x80200000`, compatível com o OpenSBI.
- O símbolo `_start` é o ponto inicial de execução.
- A seção `.bss` é zerada pelo código em `start.S`.

Capítulo 5

Boot do Sistema

O arquivo `boot/start.S` contém o primeiro código executado pelo processador após o OpenSBI transferir o controle para o microkernel. Esse arquivo é responsável por preparar o ambiente mínimo necessário para que o código em C possa ser executado corretamente.

5.1 Código Completo do `start.S`

```
.section .text.boot
.global _start

_start:
    # Inicializa gp
    la gp, __global_pointer$

    # Inicializa stack
    la sp, kernel_stack_top

    # Limpa BSS
    la t0, __bss_start
    la t1, __bss_end

1:
    bgeu t0, t1, 2f
    sd zero, 0(t0)
    addi t0, t0, 8
    j 1b

2:
    call kernel_main

3:
    j 3b

.section .bss
.align 12

kernel_stack:
    .space 4096

kernel_stack_top:
```

5.2 Visão Geral do Processo de Boot

O fluxo de inicialização é:

1. OpenSBI carrega o kernel em 0x80200000;
2. OpenSBI transfere o controle para `_start`;
3. `start.S` configura o ambiente;
4. `kernel_main()` é executado;
5. O scheduler inicia as tasks.

5.3 Diretiva `.section .text.boot`

```
.section .text.boot
```

Define uma seção específica para o código de boot.

No linker script:

```
.text : {  
    *(.text.boot)  
    *(.text*)  
}
```

Assim, o código em `.text.boot` é colocado no início da seção `.text`.

5.4 Diretiva `.global _start`

```
.global _start
```

Torna o símbolo `_start` visível ao linker.

O linker utiliza esse símbolo porque o arquivo `linker.ld` contém:

```
ENTRY(_start)
```

5.5 Rótulo `_start`

```
_start:
```

Marca o ponto inicial de execução do kernel.

5.6 Inicialização do Registrador `gp`

```
la gp, __global_pointer$
```

5.6.1 O que é o Registrador `gp`?

O registrador `gp` (`x3`) é o *Global Pointer*.

Ele é utilizado pelo compilador para acessar variáveis globais pequenas com instruções mais curtas e eficientes.

5.6.2 Definição no Linker Script

No arquivo `linker.ld`:

```
__global_pointer$ = . + 0x800;
```

5.6.3 Por que Isso é Importante?

Sem inicializar `gp`, acessos a variáveis globais podem utilizar endereços incorretos, causando falhas logo no início do boot.

5.7 Inicialização da Stack

```
la sp, kernel_stack_top
```

5.7.1 O que é o Registrador `sp`?

O registrador `sp` (x2) aponta para o topo da pilha.

A pilha é usada para:

- chamadas de função;
- armazenamento de variáveis locais;
- salvamento de registradores.

5.7.2 Por que Usar o Topo da Stack?

Em RISC-V, a stack cresce para endereços menores.

Portanto, o registrador `sp` deve apontar para o final da área reservada.

5.8 Reserva da Stack

```
.section .bss
.align 12

kernel_stack:
    .space 4096

kernel_stack_top:
```

5.8.1 `.space 4096`

Reserva 4096 bytes (4 KiB) para a stack do kernel.

5.8.2 `kernel_stack_top`

Representa o endereço imediatamente após a região reservada.

5.9 Alinhamento com `.align 12`

```
.align 12
```

Alinha o endereço em um múltiplo de 2^{12} , ou seja, 4096 bytes.

5.10 Zeragem da Seção `.bss`

5.10.1 Motivação

Variáveis globais não inicializadas são colocadas na seção `.bss`.

Pela linguagem C, essas variáveis devem iniciar com valor zero.

5.10.2 Símbolos Fornecidos pelo Linker

```
__bss_start = .;
...
__bss_end = .;
```

5.10.3 Código

```
la t0, __bss_start
la t1, __bss_end

1:
    bgeu t0, t1, 2f
    sd zero, 0(t0)
    addi t0, t0, 8
    j 1b
```

5.10.4 Explicação

- `t0 (x5)` aponta para a posição atual.
- `t1 (x6)` aponta para o final da `.bss`.
- `sd zero, 0(t0)` grava zero em 8 bytes.
- `addi t0, t0, 8` avança para a próxima posição.

5.11 Rótulos Locais Numéricos

- `1:` define um rótulo.
- `1b:` referência ao rótulo 1 anterior.
- `2f:` referência ao próximo rótulo 2.

5.12 Chamada de `kernel_main()`

```
call kernel_main
```

A pseudo-instrução `call` armazena o endereço de retorno em `ra` e transfere o controle para o código C.

5.13 Loop Infinito de Segurança

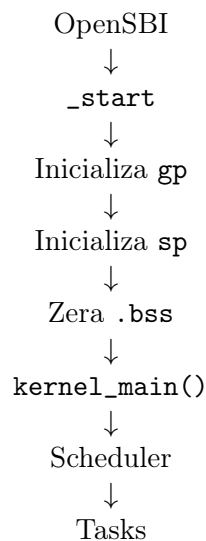
```
3:
    j 3b
```

Se `kernel_main()` retornar, o processador permanece em loop infinito.

5.14 Registradores Utilizados

Registrador	Nome ABI	Função
x1	ra	Return Address
x2	sp	Stack Pointer
x3	gp	Global Pointer
x5	t0	Ponteiro atual da BSS
x6	t1	Final da BSS
x0	zero	Constante zero

5.15 Fluxo de Execução



5.16 Por que start.S é Essencial?

Antes de qualquer código em C ser executado, é necessário garantir que:

- a pilha esteja válida;
- variáveis globais estejam corretamente inicializadas;
- o registrador `gp` esteja configurado.

Essas tarefas não são realizadas automaticamente em um sistema bare-metal.

5.17 Resumo Final

O arquivo `start.S`:

1. recebe o controle do `OpenSBI`;
2. inicializa `gp`;
3. inicializa `sp`;
4. zera a seção `.bss`;
5. chama `kernel_main()`;
6. mantém o processador em loop caso o kernel retorne.

Esse arquivo é o ponto de partida de todo o sistema operacional.

Capítulo 6

Driver UART

O módulo UART é o principal mecanismo de saída do microkernel. Ele permite que o kernel e as tasks enviem mensagens ao terminal do QEMU, o que é fundamental para depuração e observação do comportamento do sistema.

No projeto `BasicMicrokernel`, a implementação do driver UART foi intencionalmente mantida simples e direta, com foco em fins didáticos.

6.1 Arquivos do Módulo

O módulo é composto por dois arquivos:

- `include/uart.h`
- `kernel/uart.c`

6.2 Arquivo `uart.h`

6.2.1 Código Completo

```
#ifndef UART_H
#define UART_H

void uart_putc(char c);
void uart_print(const char *s);

#endif
```

6.3 Objetivo do Header

O arquivo `uart.h` declara a interface pública do driver.

Ele informa ao compilador que existem duas funções disponíveis:

- `uart_putc()`: envia um único caractere.
- `uart_print()`: envia uma string.

6.4 Diretivas de Inclusão

6.4.1 Include Guard

```
#ifndef UART_H
#define UART_H
...
#endif
```

Esse mecanismo impede que o arquivo seja processado múltiplas vezes na mesma unidade de compilação.

6.5 Arquivo `uart.c`

6.5.1 Código Completo

```
#include "uart.h"

#define UART0 0x10000000L

void uart_putc(char c)
{
    *(volatile char *)UART0 = c;
}

void uart_print(const char *s)
{
    while (*s)
    {
        uart_putc(*s++);
    }
}
```

6.6 Mapeamento de Memória

Diferentemente de programas tradicionais, o kernel acessa diretamente os registradores de hardware.

Na máquina virt do QEMU, a UART 16550A está mapeada no endereço:

`0x10000000`

Qualquer byte escrito nesse endereço é transmitido para o console.

6.7 Definição da Constante

```
#define UART0 0x10000000L
```

6.7.1 Por que usar L?

O sufixo L indica que a constante é do tipo `long`, evitando conversões implícitas inadequadas em sistemas de 64 bits.

6.8 Função `uart_putc()`

6.8.1 Código

```
void uart_putc(char c)
{
    *(volatile char *)UART0 = c;
}
```

6.8.2 Objetivo

Enviar um único caractere ao terminal.

6.8.3 Conversão de Endereço em Ponteiro

A expressão:

```
(volatile char *)UART0
```

transforma o endereço numérico em um ponteiro para um byte.

6.8.4 Uso de `volatile`

O qualificador `volatile` informa ao compilador que aquele endereço representa hardware.

Assim, cada escrita deve realmente ocorrer, sem otimizações.

6.8.5 Desreferenciação

O operador:

```
*(volatile char *)UART0
```

representa o conteúdo armazenado no endereço `0x10000000`.

6.8.6 Escrita do Byte

Ao executar:

```
*(volatile char *)UART0 = c;
```

o valor do caractere é enviado à UART.

6.9 Exemplo Prático

```
uart_putc('A');
```

Resultado no terminal:

```
A
```

6.10 Função `uart_print()`

6.10.1 Código

```
void uart_print(const char *s)
{
    while (*s)
    {
        uart_putc(*s++);
    }
}
```

6.10.2 Objetivo

Transmitir uma string terminada por `\0`.

6.11 Strings em C

Em C, uma string é um vetor de caracteres finalizado pelo byte nulo.

Exemplo:

```
"Task 1\n"
```

Representação em memória:

T a s k 1 \n \0

6.12 Funcionamento do Loop

```
while (*s)
```

O loop continua enquanto o caractere atual for diferente de zero.

6.13 Expressão `*s++`

A expressão:

```
*s++
```

equivale a:

1. ler o caractere apontado por `s`;
2. incrementar o ponteiro para o próximo caractere.

6.14 Exemplo de Execução

Considere:

```
uart_print("Hi");
```

Passos:

1. `s` aponta para `'H'`.
2. `uart_putc('H')`.

3. `s` avança.
4. `uart_putc('i')`.
5. `s` avança.
6. `*s == 0`, encerrando o loop.

6.15 Integração com o QEMU

Ao executar:

```
qemu-system-riscv64 \  
-machine virt \  
-nographic \  
-bios default \  
-kernel kernel.elf
```

o QEMU conecta a UART virtual ao terminal.

Assim, qualquer byte escrito em `0x10000000` aparece na tela.

6.16 Exemplo no Kernel

```
uart_print("=== Kernel Start ===\n");
```

Saída:

```
=== Kernel Start ===
```

6.17 Por que Não Usar `printf()`?

O kernel é compilado com:

```
-ffreestanding -nostdlib
```

Portanto, funções da biblioteca padrão não estão disponíveis.

O driver UART fornece a infraestrutura mínima de saída.

6.18 Vantagens da Implementação

- Simplicidade.
- Código extremamente pequeno.
- Independência de bibliotecas externas.
- Ideal para depuração.

6.19 Limitações

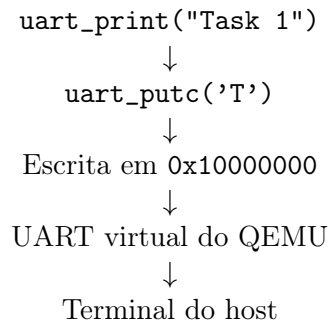
A implementação não:

- verifica se a UART está pronta;
- usa buffers;

- trata recepção;
- oferece formatação.

No ambiente do QEMU, essa simplificação funciona adequadamente.

6.20 Fluxo Completo de Impressão



6.21 Resumo Final

O módulo UART implementa a funcionalidade mais básica e importante de qualquer kernel educacional: a capacidade de imprimir mensagens. Seu funcionamento é baseado em memória mapeada em hardware, utilizando escrita direta no endereço `0x10000000`. Essa abordagem substitui o uso de `printf()` e fornece ao sistema uma interface simples e eficiente para depuração e monitoramento.

Capítulo 7

Gerência de Memória

O módulo de memória do projeto `BasicMicrokernel` implementa um alocador extremamente simples, porém suficiente para demonstrar os conceitos fundamentais de gerência dinâmica de memória em um sistema operacional. Seu objetivo principal é permitir que o kernel reserve regiões de memória em tempo de execução, por exemplo para stacks de tasks.

7.1 Objetivo do Módulo

O módulo oferece três funcionalidades principais:

- inicializar o heap do kernel;
- alocar blocos de memória sequencialmente;
- informar a quantidade de memória utilizada.

Esse modelo é conhecido como *bump allocator* ou *linear allocator*.

7.2 Arquivos do Módulo

- `include/memory.h`
- `kernel/memory.c`

7.3 Arquivo `memory.h`

7.3.1 Código Completo

```
#ifndef MEMORY_H
#define MEMORY_H

#include <stdint.h>

void memory_init(void);
void *kmalloc(uint64_t size);
uint64_t memory_used(void);

#endif
```

7.4 Funções Declaradas

Função	Objetivo
<code>memory_init()</code>	Inicializa o heap
<code>kmalloc()</code>	Aloca um bloco de memória
<code>memory_used()</code>	Retorna o total utilizado

7.5 Arquivo `memory.c`

7.5.1 Código Completo

```
#include "memory.h"

#define HEAP_SIZE 0x10000

static uint8_t heap[HEAP_SIZE];
static uint64_t heap_top = 0;

void memory_init(void)
{
    heap_top = 0;
}

void *kmalloc(uint64_t size)
{
    /* Alinhamento para 8 bytes */
    size = (size + 7) & ~7ULL;

    if (heap_top + size > HEAP_SIZE)
        return 0;

    void *ptr = &heap[heap_top];
    heap_top += size;

    return ptr;
}

uint64_t memory_used(void)
{
    return heap_top;
}
```

7.6 Conceito de Heap

O heap é uma área de memória utilizada para alocação dinâmica.

Diferentemente da stack, cuja alocação e desalocação ocorrem automaticamente, o heap é controlado explicitamente pelo sistema operacional.

7.7 Definição do Heap

```
#define HEAP_SIZE 0x10000
static uint8_t heap[HEAP_SIZE];
```

- $0x10000 = 65536 \text{ bytes} = 64 \text{ KiB}$.

- `heap[]` é um vetor estático localizado na seção `.bss`.

7.8 Ponteiro de Alocação

```
static uint64_t heap_top = 0;
```

Essa variável indica o deslocamento do próximo bloco livre dentro do heap.

7.9 Visualização Inicial

<code>heap[0]</code>	início do heap
<code>heap_top = 0</code>	próxima posição livre

7.10 Função `memory_init()`

```
void memory_init(void)
{
    heap_top = 0;
}
```

Reinicia o alocador, marcando todo o heap como livre.

7.11 Quando é Chamada?

Tipicamente em:

```
void kernel_main(void)
{
    memory_init();
    ...
}
```

7.12 Função `kmalloc()`

```
void *kmalloc(uint64_t size)
{
    size = (size + 7) & ~7ULL;

    if (heap_top + size > HEAP_SIZE)
        return 0;

    void *ptr = &heap[heap_top];
    heap_top += size;

    return ptr;
}
```

7.13 Passo 1: Alinhamento para 8 Bytes

```
size = (size + 7) & ~7ULL;
```

7.13.1 Motivação

A arquitetura RV64 trabalha naturalmente com palavras de 64 bits (8 bytes). Alinhar as alocações melhora desempenho e evita acessos desalinhados.

7.13.2 Exemplos

Valor solicitado	Valor alinhado
1	8
8	8
9	16
15	16
17	24

7.13.3 Explicação Bit a Bit

- Soma-se 7 ao valor.
- Limpa-se os três bits menos significativos.

7.14 Passo 2: Verificação de Espaço

```
if (heap_top + size > HEAP_SIZE)
    return 0;
```

Se não houver espaço suficiente, a função retorna NULL.

7.15 Passo 3: Definição do Ponteiro

```
void *ptr = &heap[heap_top];
```

O ponteiro retornado aponta para o início do bloco alocado.

7.16 Passo 4: Avanço do Heap

```
heap_top += size;
```

A próxima alocação ocorrerá após o bloco recém-reservado.

7.17 Passo 5: Retorno

```
return ptr;
```

7.18 Função memory_used()

```
uint64_t memory_used(void)
{
    return heap_top;
}
```

Retorna o total de bytes já consumidos no heap.

7.19 Exemplo Prático

```
memory_init();

void *a = kmalloc(1000);
void *b = kmalloc(500);

uart_print("Memoria usada: ");
uart_print_dec(memory_used());
uart_print(" bytes\n");
```

Saída aproximada:

```
Memoria usada: 1504 bytes
```

(1000 é alinhado para 1000, 500 é alinhado para 504.)

7.20 Uso na Criação de Tasks

No arquivo `task.c`:

```
t->stack = (uint8_t*) kmalloc(stack_size);
```

Cada task recebe sua própria stack a partir do heap.

7.21 Representação Visual

Task 1 Stack	Task 2 Stack	Task 3 Stack	Livre
--------------	--------------	--------------	-------

7.22 Características do Bump Allocator

- extremamente simples;
- alta velocidade;
- não possui fragmentação interna complexa;
- não permite liberação individual.

7.23 Limitações

O alocador não implementa:

- `kfree()`;
- reutilização de blocos;
- coalescência;
- proteção contra concorrência.

7.24 Complexidade

- `memory_init()`: $O(1)$
- `kmalloc()`: $O(1)$
- `memory_used()`: $O(1)$

7.25 Por que Essa Implementação é Adequada?

O objetivo do projeto é educacional. O foco é mostrar:

- como a memória pode ser reservada dinamicamente;
- como tasks obtêm stacks independentes;
- como o kernel controla recursos sem bibliotecas externas.

7.26 Resumo Final

O módulo `memory.c` implementa um alocador linear baseado em um heap estático de 64 KiB.

Cada chamada a `kmalloc()` reserva um novo bloco e avança o ponteiro `heap_top`. Essa abordagem simples é suficiente para a criação de stacks de tasks e para ilustrar os princípios fundamentais de gerência de memória em sistemas operacionais.

Capítulo 8

Gerenciamento de Tasks: Análise Detalhada de `task.h` e `task.c`

O módulo de tasks é responsável por representar, criar e inicializar as unidades básicas de execução do microkernel.

Cada task possui:

- seu próprio contexto de registradores;
- um contador de programa (`sepc`);
- uma stack privada;
- prioridade;
- ponteiro para a função de entrada.

Esse módulo estabelece a base sobre a qual o escalonador e o mecanismo de troca de contexto operam.

8.1 Conceito de Task

Uma task (ou thread) representa um fluxo independente de execução.

Embora todas compartilhem o mesmo espaço de endereçamento, cada task mantém seu próprio estado de CPU.

Esse estado inclui:

- conteúdo dos registradores;
- endereço da próxima instrução;
- stack.

Quando o scheduler seleciona uma task, esse estado é restaurado e a execução prossegue exatamente do ponto onde havia sido interrompida.

8.2 Arquivos do Módulo

- `include/task.h`
- `kernel/task.c`

8.3 Arquivo task.h

8.3.1 Código Completo

```
#ifndef TASK_H
#define TASK_H

#include <stdint.h>

#define MAX_TASKS 8

typedef struct
{
    uint64_t regs[31];
    uint64_t sepc;

    void (*entry)(void);

    int priority;
    int state;

    uint8_t *stack;
    uint64_t stack_size;
} TCB;

extern TCB tasks[MAX_TASKS];
extern int task_count;

void xTaskCreate(void (*task)(void),
                uint64_t stack_size,
                int priority);

#endif
```

8.4 Constante MAX_TASKS

```
#define MAX_TASKS 8
```

Define o número máximo de tasks simultâneas.

O valor 8 é suficiente para fins didáticos e simplifica a implementação ao evitar alocação dinâmica do vetor de TCBs.

8.5 Estrutura TCB

TCB significa *Task Control Block*.

Essa estrutura contém todas as informações necessárias para representar uma task.

8.5.1 regs[31]

```
uint64_t regs[31];
```

Armazena os registradores gerais da CPU.

Como o RISC-V possui 32 registradores inteiros (x0 a x31) e x0 é sempre zero, apenas 31 registradores precisam ser salvos.

Mapeamento

Índice	Registrador	Nome ABI
0	x1	ra
1	x2	sp
2	x3	gp
3	x4	tp
4	x5	t0
⋮	⋮	⋮
30	x31	t6

8.5.2 sepc

```
uint64_t sepc;
```

Armazena o valor do registrador CSR **sepc** (*Supervisor Exception Program Counter*).

Esse registrador contém o endereço da próxima instrução a ser executada ao retornar de um trap com **sret**.

8.5.3 entry

```
void (*entry)(void);
```

Ponteiro para a função que implementa a task.

Exemplo:

```
void task1(void)
{
    while (1)
        uart_print("Task 1\n");
}
```

8.5.4 priority

```
int priority;
```

Armazena a prioridade atribuída à task.

No Round-Robin puro esse campo não é utilizado, mas foi incluído para permitir a implementação futura de escalonamento por prioridades.

8.5.5 state

```
int state;
```

Representa o estado da task.

Exemplos possíveis:

- READY
- RUNNING
- BLOCKED

- SUSPENDED

No projeto atual esse campo é reservado para extensões futuras.

8.5.6 stack

```
uint8_t *stack;
```

Ponteiro para a base da stack da task.

8.5.7 stack_size

```
uint64_t stack_size;
```

Quantidade de bytes reservados para a stack.

8.6 Variáveis Globais

```
extern TCB tasks[MAX_TASKS];
extern int task_count;
```

- tasks: vetor contendo todos os TCBs.
- task_count: número de tasks já criadas.

8.7 Função xTaskCreate()

A interface foi inspirada no FreeRTOS, porém simplificada.

```
void xTaskCreate(void (*task)(void),
                uint64_t stack_size,
                int priority);
```

Parâmetros:

- task: função de entrada.
- stack_size: tamanho da stack.
- priority: prioridade.

8.8 Arquivo task.c

8.8.1 Código Completo

```
#include "task.h"
#include "memory.h"

TCB tasks[MAX_TASKS];
int task_count = 0;

void xTaskCreate(void (*task)(void),
                uint64_t stack_size,
                int priority)
{
    if (task_count >= MAX_TASKS)
        return;
```



```

TCB *t = &tasks[task_count++];

t->entry = task;
t->priority = priority;
t->state = 0;
t->stack_size = stack_size;

t->stack = (uint8_t *) kmalloc(stack_size);
if (!t->stack)
    return;

uint64_t *sp =
    (uint64_t *) (t->stack + stack_size);

t->regs[0] = (uint64_t) task; /* ra */
t->regs[1] = (uint64_t) sp; /* sp */
t->sepc = (uint64_t) task;
}

```

8.9 Definição das Variáveis Globais

```

TCB tasks[MAX_TASKS];
int task_count = 0;

```

Essas variáveis são alocadas na seção `.bss` e inicializadas com zero pelo código de boot.

8.10 Validação do Número Máximo

```

if (task_count >= MAX_TASKS)
    return;

```

Impede que o vetor `tasks[]` seja acessado fora dos limites.

8.11 Seleção do Próximo TCB

```

TCB *t = &tasks[task_count++];

```

- `tasks[task_count]` seleciona a próxima posição livre.
- O operador pós-incremento aumenta `task_count`.

8.12 Inicialização de Campos

```

t->entry = task;
t->priority = priority;
t->state = 0;
t->stack_size = stack_size;

```

Armazena as informações básicas da task.

8.13 Alocação da Stack

```
t->stack = (uint8_t *) kmalloc(stack_size);
```

Solicita ao alocador de memória um bloco exclusivo para a task.

8.14 Verificação de Falha

```
if (!t->stack)
    return;
```

Se o heap estiver esgotado, a criação da task falha.

8.15 Inicialização do Stack Pointer

```
uint64_t *sp =
    (uint64_t *) (t->stack + stack_size);
```

Como a stack cresce para endereços menores, o ponteiro inicial deve apontar para o final da região alocada.

8.16 Preparação do Contexto Inicial

8.16.1 Registrador ra

```
t->regs[0] = (uint64_t) task;
```

Quando o contexto for restaurado, o registrador **ra** conterá o endereço da função da task.

8.16.2 Registrador sp

```
t->regs[1] = (uint64_t) sp;
```

Define o valor inicial da stack.

8.16.3 Registrador sepc

```
t->sepc = (uint64_t) task;
```

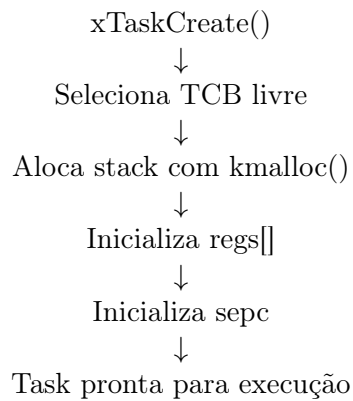
Quando um retorno via **sret** ocorrer, a execução iniciará na função da task.

8.17 Por que Salvar ra e sepc?

O projeto suporta dois mecanismos de troca de contexto:

- **Cooperativo:** via `yield()` e `context_switch()`.
- **Preemptivo:** via timer e retorno com **sret**.
- **ra** é usado no contexto cooperativo.
- **sepc** é usado no contexto preemptivo.

8.18 Fluxo de Criação de Task



8.19 Exemplo de Uso

```
xTaskCreate(task1, 1024, 1);  
xTaskCreate(task2, 2048, 2);
```

Cada task receberá:

- stack independente;
- prioridade configurada;
- contexto inicial preparado.

8.20 Resumo Final

O módulo `task.c` implementa a infraestrutura básica para criação de tasks. Cada task é representada por um TCB contendo registradores, stack e metadados. A função `xTaskCreate()` aloca a stack, inicializa o contexto e prepara a task para que o scheduler possa executá-la posteriormente.

Capítulo 9

Chaveamento de Contexto

O chaveamento de contexto (*context switch*) é o mecanismo que permite ao sistema operacional interromper a execução de uma task e continuar a execução de outra task exatamente do ponto em que ela havia parado.

Esse mecanismo é a base da multitarefa. Embora exista apenas uma CPU, o kernel alterna rapidamente entre diferentes tasks, criando a impressão de execução simultânea.

9.1 Objetivo do Chaveamento de Contexto

Cada task possui um estado de execução próprio, composto por:

- endereço da próxima instrução a ser executada;
- ponteiro da pilha (**sp**);
- registradores de propósito geral;
- variáveis locais armazenadas na stack.

Ao trocar de task, o kernel deve:

1. salvar o estado da task atual;
2. restaurar o estado da próxima task;
3. continuar a execução da nova task.

9.2 Registradores RISC-V: Nome Arquitetural vs Nome ABI

Na especificação do RISC-V, os registradores são numerados de **x0** a **x31**. Na ABI (Application Binary Interface), cada registrador recebe um nome simbólico.

Número	Nome ABI	Função	Preservado?
x0	zero	Constante zero	Sim
x1	ra	Return Address	Não
x2	sp	Stack Pointer	Sim
x3	gp	Global Pointer	Sim
x4	tp	Thread Pointer	Sim
x5–x7	t0–t2	Temporários	Não
x8–x9	s0–s1	Registradores salvos	Sim
x10–x17	a0–a7	Argumentos e retorno	Não

x18–x27	s2–s11	Registradores salvos	Sim
x28–x31	t3–t6	Temporários	Não

Nos arquivos em Assembly do projeto são utilizados os nomes ABI, pois eles tornam o código mais legível.

9.3 Estrutura do Contexto no TCB

Cada task possui um vetor:

```
uint64_t regs[31];
```

Esse vetor armazena os registradores x1 a x31. O registrador x0 não é armazenado porque seu valor é sempre zero.

Offset (bytes)	Registrador
0	ra (x1)
8	sp (x2)
16	gp (x3)
24	tp (x4)
32	t0 (x5)
40	t1 (x6)
48	t2 (x7)
56	s0 (x8)
64	s1 (x9)
72	a0 (x10)
80	a1 (x11)
88	a2 (x12)
96	a3 (x13)
104	a4 (x14)
112	a5 (x15)
120	a6 (x16)
128	a7 (x17)
136	s2 (x18)
144	s3 (x19)
152	s4 (x20)
160	s5 (x21)
168	s6 (x22)
176	s7 (x23)
184	s8 (x24)
192	s9 (x25)
200	s10 (x26)
208	s11 (x27)
216	t3 (x28)
224	t4 (x29)
232	t5 (x30)
240	t6 (x31)

9.4 Assinatura da Função

```
void context_switch(void *old, void *new);
```

Na prática, os parâmetros são:

- `old`: endereço de `tasks[prev].regs`;
- `new`: endereço de `tasks[next].regs`.

9.5 Código em Assembly

```
.section .text
.global context_switch

# a0 = &old->regs
# a1 = &new->regs

context_switch:
    # Salva contexto atual
    sd ra, 0(a0)
    sd sp, 8(a0)
    sd gp, 16(a0)
    ...
    sd t6, 240(a0)

    # Carrega contexto da próxima task
    ld ra, 0(a1)
    ld sp, 8(a1)
    ld gp, 16(a1)
    ...
    ld t6, 240(a1)

    ret
```

9.6 Parâmetros em RISC-V

Na ABI RISC-V, os argumentos de uma função são passados nos registradores:

- `a0` (`x10`): primeiro argumento;
- `a1` (`x11`): segundo argumento.

Assim, ao entrar em `context_switch()`:

- `a0` aponta para o contexto antigo;
- `a1` aponta para o contexto da próxima task.

9.7 Etapa 1: Salvamento do Contexto Atual

Exemplo:

```
sd ra, 0(a0)
sd sp, 8(a0)
```

Essas instruções armazenam os valores atuais de `ra` e `sp` na estrutura da task atual.

O mesmo processo é repetido para todos os registradores.

9.8 Etapa 2: Restauração do Próximo Contexto

Exemplo:

```
ld ra, 0(a1)
ld sp, 8(a1)
```

Agora a CPU passa a conter o estado da próxima task.

9.9 Etapa 3: Execução de `ret`

A instrução:

```
ret
```

equivale a:

```
jalr x0, ra, 0
```

Ou seja, o processador salta para o endereço contido em `ra`, já restaurado a partir do contexto da nova task.

9.10 Inicialização do Contexto em `xTaskCreate()`

Ao criar uma task:

```
t->regs[0] = (uint64_t)task; // ra
t->regs[1] = (uint64_t)sp; // sp
```

Isso significa que, quando essa task for restaurada pela primeira vez:

- `ra` apontará para a função da task;
- `sp` apontará para a stack recém-alocada.

Ao executar `ret`, a task iniciará sua execução normalmente.

9.11 Exemplo Prático

Considere duas tasks:

- Task 1;
- Task 2.

Quando Task 1 chama `yield()`:

1. O scheduler seleciona Task 2.
2. `context_switch()` salva o estado de Task 1.
3. O estado de Task 2 é restaurado.
4. `ret` transfere o controle para Task 2.

Posteriormente, quando Task 2 ceder a CPU, o processo ocorre no sentido inverso.

9.12 Por que Salvar Todos os Registradores?

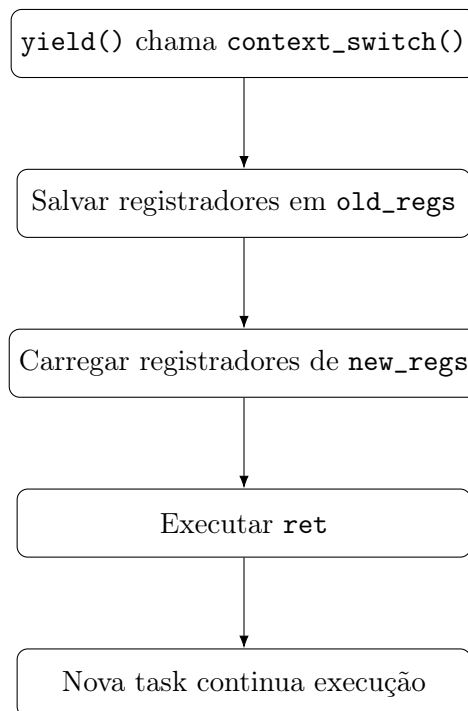
Embora a ABI exija que apenas alguns registradores sejam preservados entre chamadas de função, o kernel salva todos os registradores relevantes.

Isso garante que o chaveamento de contexto funcione corretamente independentemente do ponto em que a task foi interrompida.

9.13 Separação de Responsabilidades

- `scheduler.c`: decide qual task executar.
- `context.S`: salva e restaura os registradores.
- `task.c`: prepara o contexto inicial.

9.14 Fluxograma do Context Switch



9.15 Resumo

O arquivo `context.S` implementa o núcleo da multitarefa do sistema operacional.

Seu funcionamento consiste em:

1. copiar o estado atual da CPU para o TCB da task corrente;
2. carregar o estado armazenado no TCB da próxima task;
3. executar `ret`;
4. continuar a execução da nova task.

Esse mecanismo permite que múltiplas tasks compartilhem a CPU de forma transparente e eficiente.

Capítulo 10

Escalonador

O escalonador (*scheduler*) é o componente responsável por decidir qual task deve utilizar a CPU em cada instante.

No projeto **BasicMicrokernel**, o escalonador foi projetado com dois objetivos principais:

- Implementar o algoritmo Round-Robin.
- Permitir tanto escalonamento cooperativo quanto preemptivo.

O scheduler não executa diretamente a troca de contexto em Assembly, mas decide qual task será executada e aciona o mecanismo adequado.

10.1 Arquivos do Módulo

- `include/scheduler.h`
- `kernel/scheduler.c`

10.2 Objetivos do Módulo

O módulo fornece:

- seleção do algoritmo de escalonamento;
- troca cooperativa via `yield()`;
- troca preemptiva via `schedule_from_trap()`;
- inicialização da primeira task.

10.3 Arquivo `scheduler.h`

10.3.1 Código Completo

```
#ifndef SCHEDULER_H
#define SCHEDULER_H

#include <stdint.h>

typedef int (*sched_algo_t)(void);

void scheduler_set_algorithm(sched_algo_t algo);
void scheduler_start(void);
```

```
void yield(void);

void schedule_from_trap(uint64_t *frame);
int scheduler_current_task(void);

#endif
```

10.4 Tipo sched_algo_t

```
typedef int (*sched_algo_t)(void);
```

Esse tipo representa um ponteiro para função que:

- não recebe parâmetros;
- retorna o índice da próxima task.

Exemplo:

```
static int round_robin(void)
{
    return (current + 1) % task_count;
}
```

Essa abstração permite trocar o algoritmo sem alterar o restante do código.

10.5 Funções Públicas

Função	Objetivo
<code>scheduler_set_algorithm()</code>	Seleciona o algoritmo
<code>scheduler_start()</code>	Inicia a primeira task
<code>yield()</code>	Troca cooperativa
<code>schedule_from_trap()</code>	Troca preemptiva
<code>scheduler_current_task()</code>	Retorna o índice atual

10.6 Arquivo scheduler.c

10.6.1 Código Completo

```
#include "scheduler.h"
#include "task.h"
#include <stdint.h>

extern TCB tasks[];
extern int task_count;

extern void context_switch(void*, void*);

static int current = 0;

static int round_robin(void)
{
    if (task_count == 0)
        return 0;
}
```

```

    return (current + 1) % task_count;
}

static sched_algo_t current_algo = round_robin;

void scheduler_set_algorithm(sched_algo_t algo)
{
    if (algo)
        current_algo = algo;
}

int scheduler_current_task(void)
{
    return current;
}

void yield(void)
{
    if (task_count < 2)
        return;

    int prev = current;
    int next = current_algo();

    if (prev == next)
        return;

    current = next;

    context_switch(tasks[prev].regs,
                   tasks[next].regs);
}

void schedule_from_trap(uint64_t *frame)
{
    if (task_count < 2)
        return;

    int prev = current;
    int next = current_algo();

    if (prev == next)
        return;

    for (int i = 0; i < 31; i++)
        tasks[prev].regs[i] = frame[i];

    asm volatile(
        "csrr %0, sepc"
        : "=r"(tasks[prev].sepc)
    );

    current = next;

    for (int i = 0; i < 31; i++)
        frame[i] = tasks[next].regs[i];
}

```

```

    asm volatile(
        "csrw sepc, %0"
        :
        : "r"(tasks[next].sepc)
    );
}

void scheduler_start(void)
{
    if (task_count == 0)
        return;

    current = 0;
    tasks[0].entry();
}

```

10.7 Variáveis Globais

10.7.1 current

```
static int current = 0;
```

Armazena o índice da task em execução.

10.7.2 current_algo

```
static sched_algo_t current_algo = round_robin;
```

Ponteiro para o algoritmo atualmente selecionado.

10.8 Algoritmo Round-Robin

```
static int round_robin(void)
{
    return (current + 1) % task_count;
}

```

Seleciona a próxima task em ordem circular.

Exemplo com três tasks:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 0$$

10.9 Função scheduler_set_algorithm()

```
void scheduler_set_algorithm(sched_algo_t algo)
{
    if (algo)
        current_algo = algo;
}

```

Permite substituir o algoritmo padrão por outro.

Exemplo:

```
scheduler_set_algorithm(priority_scheduler);
```

10.10 Função scheduler_current_task()

```
int scheduler_current_task(void)
{
    return current;
}
```

Retorna o índice da task atualmente em execução.

10.11 Função yield()

10.11.1 Objetivo

Realizar troca de contexto cooperativa.

10.11.2 Código

```
void yield(void)
{
    if (task_count < 2)
        return;

    int prev = current;
    int next = current_algo();

    if (prev == next)
        return;

    current = next;

    context_switch(tasks[prev].regs,
                  tasks[next].regs);
}
```

10.11.3 Fluxo

1. A task chama yield().
2. O algoritmo seleciona a próxima task.
3. context_switch() salva e restaura registradores.
4. A nova task passa a executar.

10.12 Função schedule_from_trap()

10.12.1 Objetivo

Realizar troca de contexto preemptiva após interrupção de timer.

10.12.2 Parâmetro

```
uint64_t *frame
```

Ponteiro para os registradores salvos por `trap_entry.S`.

10.12.3 Etapas

1. Salva o frame no TCB atual.
2. Lê o CSR `sepc`.
3. Seleciona a próxima task.
4. Copia o contexto da nova task para o frame.
5. Atualiza o CSR `sepc`.

10.12.4 Por que não usar `context_switch()`?

Durante uma interrupção, os registradores já estão salvos na stack do trap handler. Portanto, basta manipular o frame e o registrador `sepc`.

10.13 Função `scheduler_start()`

```
void scheduler_start(void)
{
    if (task_count == 0)
        return;

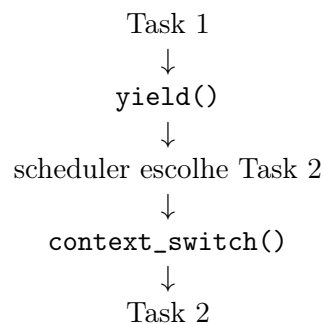
    current = 0;
    tasks[0].entry();
}
```

Inicializa a execução da primeira task.

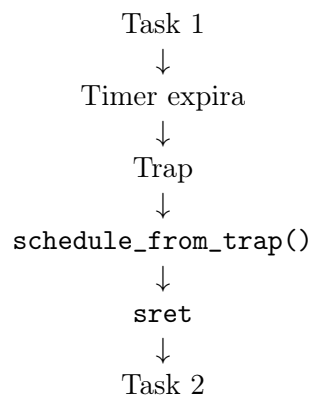
A partir desse ponto:

- `yield()` possibilita multitarefa cooperativa;
- o timer permite multitarefa preemptiva.

10.14 Fluxo Cooperativo



10.15 Fluxo Preemptivo



10.16 Separação de Responsabilidades

Módulo	Responsabilidade
task.c	Armazenar contextos
scheduler.c	Escolher próxima task
context.S	Troca cooperativa
trap.c	Tratamento de interrupções
timer.c	Programação do timer

10.17 Resumo Final

O módulo `scheduler.c` é o núcleo lógico da multitarefa. Ele encapsula a política de escalonamento e suporta dois mecanismos de ativação:

- cooperativo, via `yield()`;
- preemptivo, via interrupções de timer.

O uso de um ponteiro para função (`sched_algo_t`) torna o sistema extensível e permite a implementação futura de algoritmos mais sofisticados, como escalonamento por prioridades ou EDF.

Capítulo 11

Timer

Na versão atual do projeto, o escalonamento é exclusivamente cooperativo. Isso significa que a troca de contexto ocorre apenas quando a própria task chama explicitamente a função `yield()`. Por esse motivo, o projeto ainda não possui os arquivos `timer.c` e `timer.h`. Não há, portanto, um mecanismo de interrupção periódica para forçar a troca de contexto automaticamente.

11.1 Por que o Timer Ainda Não Foi Implementado?

A implementação do timer exige recursos adicionais:

- configuração do registrador `stvec`;
- tratamento de traps;
- uso da interface SBI fornecida pelo OpenSBI;
- salvamento e restauração de contexto em interrupções.

Como o objetivo desta versão é demonstrar os fundamentos da multitarefa, optou-se por implementar inicialmente apenas o modelo cooperativo.

11.2 Escalonamento Cooperativo vs. Preemptivo

Característica	Cooperativo	Preemptivo
Troca por <code>yield()</code>	Sim	Opcional
Interrupções de timer	Não	Sim
Arquivos <code>timer.c/h</code>	Não existem	Necessários
Arquivos <code>trap.c</code>	Não existem	Necessários

11.3 Como Seria a Evolução

Em versões futuras, o timer poderá ser implementado para:

1. programar interrupções periódicas;
2. chamar o escalonador automaticamente;
3. permitir multitarefa preemptiva.

11.4 Resumo

Embora o projeto ainda não possua suporte a timer, sua arquitetura já está preparada para receber esse recurso futuramente. O escalonador e o contexto das tasks foram projetados de forma modular para facilitar essa evolução.

Capítulo 12

Traps e Interrupções

Na versão atual do microkernel, não existe tratamento de traps. Consequentemente, os arquivos `trap.c`, `trap.h` e `trap_entry.S` ainda não fazem parte do projeto.

12.1 O que é um Trap?

Na especificação RISC-V, trap é qualquer evento que interrompe o fluxo normal de execução e transfere o controle para um tratador no sistema operacional. No contexto de RISC-V (e de várias arquiteturas modernas), o termo trap é mais abrangente do que apenas “erro”.

Ou seja, trap é um termo genérico que engloba:

- Interrupções (interrupts);
- Exceções (exceptions);
- Chamadas de sistema (system calls).

12.2 Classificação Oficial no RISC-V

A especificação divide traps em duas categorias:

12.2.1 Interrupções (Interrupts)

Eventos assíncronos, causados por hardware externo ou temporizadores. Exemplos:

- Timer interrupt;
- UART interrupt;
- External interrupt.

12.2.2 Exceções (Exceptions)

Eventos síncronos, causados pela instrução em execução. Exemplos:

- Instrução ilegal;
- Divisão por zero (em algumas arquiteturas);
- Page fault;
- ecall (system call).

A instrução ecall é Trap, mas Não é Erro. Quando um programa em user mode executa:

O processador gera uma exceção síncrona. Isso é um trap legítimo e esperado, não um erro. Timer Interrupt Também é Trap e Quando o timer expira:

- o hardware sinaliza uma interrupção;
- o processador salva o contexto;
- o controle é transferido para stvec.

Esse evento também é classificado como trap.

12.3 Motivação

Como o sistema utiliza escalonamento cooperativo, todas as trocas de contexto ocorrem de forma voluntária por meio da função `yield()`. Dessa forma, o kernel não precisa interceptar eventos assíncronos do hardware.

12.4 O que é um Trap?

Em RISC-V, um trap é qualquer evento que desvia a execução normal do programa para um tratador especial. Exemplos:

- interrupções de timer;
- exceções de acesso inválido à memória;
- instruções `ecall`;
- instruções ilegais.

12.5 Quando o Suporte a Traps Será Necessário?

O tratamento de traps torna-se essencial para:

- multitarefa preemptiva;
- system calls;
- isolamento entre kernel e user mode;
- tratamento de exceções.

12.6 Funcionamento Atual do Escalonamento

O fluxo atual é inteiramente cooperativo:

1. Uma task executa normalmente.
2. A task chama `yield()`.
3. O scheduler seleciona a próxima task.
4. `context_switch()` realiza a troca de registradores.
5. A nova task continua sua execução.

12.7 Possível Evolução

Em uma versão futura, a arquitetura poderá ser estendida com:

- `trap_entry.S` para salvar registradores;
- `trap.c` para identificar a causa do trap;
- integração com `timer.c`;
- retorno via `sret`.

12.8 Resumo

A ausência de suporte a traps é intencional e compatível com o objetivo didático desta etapa do projeto. Mesmo sem interrupções, o kernel já demonstra claramente os conceitos fundamentais de multitarefa, escalonamento e chaveamento de contexto.

Capítulo 13

Compilação e Execução

13.1 Compilação

```
make
```

13.2 Execução

```
qemu-system-riscv64 \  
-machine virt \  
-m 128M \  
-nographic \  
-bios default \  
-kernel kernel.elf
```

13.3 Finalizar Qemu no Terminal

Pressione as combinações e teclas conform a seguir:

```
Ctrl+a  
x
```

13.4 Saída Esperada

```
=== Preemptive Kernel ===  
Task 1  
Task 2  
Task 1  
Task 2  
...
```

Capítulo 14

Tutorial de Implementação do Escalonamento Preemptivo

Este capítulo apresenta um roteiro passo a passo para evoluir o `BasicMicrokernel` de um sistema com escalonamento cooperativo para um sistema com escalonamento preemptivo baseado em interrupções periódicas de timer. O objetivo é descrever:

- quais arquivos devem ser criados ou modificados;
- quais funções devem ser implementadas;
- qual a responsabilidade de cada função;
- como os módulos interagem entre si.

Nenhum código completo é apresentado; apenas protótipos e a lógica necessária para implementação.

14.1 Objetivo Final

Ao concluir este tutorial, o kernel será capaz de:

- gerar interrupções periódicas de timer;
- tratar traps em modo supervisor;
- salvar e restaurar o contexto das tasks;
- alternar automaticamente entre tasks;
- utilizar o algoritmo Round-Robin.

14.2 Arquivos Necessários

14.2.1 Novos Arquivos

- `include/timer.h`
- `kernel/timer.c`
- `include/trap.h`
- `kernel/trap.c`
- `boot/trap_entry.S`

14.2.2 Arquivos a Modificar

- `kernel/main.c`
- `include/scheduler.h`
- `kernel/scheduler.c`
- `Makefile`

14.3 Etapa 1: Implementação do Timer

O timer é responsável por gerar interrupções periódicas que forçam a execução do escalonador.

14.3.1 Arquivo `timer.h`

Declarar as funções públicas:

```
void timer_init(uint64_t interval);  
void timer_next(void);
```

14.3.2 Função `timer_init()`

Responsabilidades:

1. armazenar o intervalo desejado;
2. programar o primeiro evento de timer;
3. habilitar a interrupção STIE no CSR `sie`;
4. habilitar interrupções globais no CSR `sstatus`.

14.3.3 Função `timer_next()`

Responsabilidades:

1. ler o CSR `time`;
2. somar o intervalo configurado;
3. invocar a SBI para programar o próximo evento.

14.3.4 Integração com OpenSBI

A programação do timer é realizada por meio de uma chamada `ecall` utilizando a extensão legacy `set_timer` do SBI.

14.4 Etapa 2: Implementação do Trap Handler

O trap handler recebe todos os traps (interrupções e exceções) e identifica a causa.

14.4.1 Arquivo `trap.h`

Declarar:

```
void trap_handler(uint64_t *frame);
```

14.4.2 Parâmetro frame

O parâmetro aponta para o conjunto de registradores salvos em `trap_entry.S`.

Seu layout deve coincidir exatamente com o vetor:

```
uint64_t regs[31];
```

presente na estrutura TCB.

14.4.3 Função `trap_handler()`

Responsabilidades:

1. ler o CSR `scause`;
2. verificar se o trap é interrupção ou exceção;
3. identificar interrupção de timer;
4. chamar `timer_next()`;
5. chamar `schedule_from_trap(frame)`.

14.5 Etapa 3: Implementação de `trap_entry.S`

Este arquivo é o ponto de entrada definido em `stvec`.

14.5.1 Responsabilidades

1. reservar espaço na stack;
2. salvar registradores `x1` a `x31`;
3. passar o ponteiro do frame em `a0`;
4. chamar `trap_handler()`;
5. restaurar os registradores;
6. liberar a stack;
7. executar `sret`.

14.6 Etapa 4: Extensão do Scheduler

O scheduler já implementa o algoritmo Round-Robin para o modo cooperativo.

Agora ele deve também operar em modo preemptivo.

14.6.1 Arquivo `scheduler.h`

Adicionar:

```
void schedule_from_trap(uint64_t *frame);
```

14.6.2 Função `schedule_from_trap()`

Responsabilidades:

1. salvar o frame no TCB da task atual;

2. salvar o CSR `sepc`;
3. selecionar a próxima task usando o algoritmo atual;
4. copiar o contexto da próxima task para o frame;
5. restaurar o CSR `sepc`.

14.7 Etapa 5: Reutilização do Round-Robin

O algoritmo já existente pode continuar sendo utilizado:

```
static int round_robin(void);
```

Ele retorna o índice da próxima task a executar.

O mesmo algoritmo será utilizado tanto por:

- `yield()`;
- `schedule_from_trap()`.

14.8 Etapa 6: Configuração do `main.c`

No arquivo `main.c`, o kernel deve:

1. criar as tasks;
2. configurar o CSR `stvec`;
3. inicializar o timer;
4. iniciar o scheduler.

14.8.1 Declaração Necessária

```
extern void trap_entry(void);
```

14.8.2 Configuração do Vetor de Trap

```
asm volatile("csrw stvec, %0" :: "r"(trap_entry));
```

14.8.3 Inicialização do Timer

```
timer_init(interval);
```

14.9 Etapa 7: Atualização do Makefile

Adicionar os seguintes objetos:

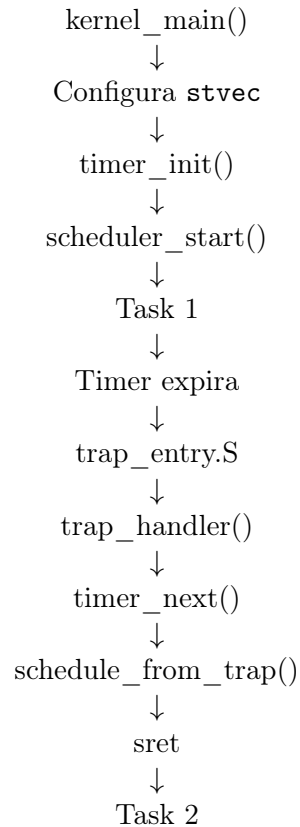
- `trap_entry.o`
- `trap.o`
- `timer.o`

14.10 Funcionamento do Round-Robin

O Round-Robin seleciona circularmente a próxima task.

$0 \rightarrow 1 \rightarrow 2 \rightarrow 0 \rightarrow 1 \rightarrow 2$

14.11 Fluxo Completo de Execução



14.12 Teste do Sistema

Remova todas as chamadas a `yield()` das tasks.

Se as tasks continuarem alternando automaticamente, o sistema está operando em modo preemptivo.

14.13 Checklist Final

14.13.1 Timer

- Criar `timer.h`
- Criar `timer.c`
- Implementar `timer_init()`
- Implementar `timer_next()`

14.13.2 Trap

- Criar `trap.h`

- Criar `trap.c`
- Criar `trap_entry.S`
- Implementar `trap_handler()`

14.13.3 Scheduler

- Declarar `schedule_from_trap()`
- Implementar `schedule_from_trap()`

14.13.4 Main

- Configurar `stvec`
- Chamar `timer_init()`

14.13.5 Build

- Atualizar o `Makefile`

14.14 Resultado Esperado

Após a implementação, o kernel deverá alternar automaticamente entre as tasks sem depender de chamadas explícitas a `yield()`. O sistema passará a utilizar:

Escalonamento Preemptivo + Round-Robin

Capítulo 15

Tutorial Guiado: Fluxo de Implementação do Timer, Trap e Escalonamento Round-Robin

Este capítulo apresenta o esqueleto de código para implementar o escalonamento preemptivo no `BasicMicrokernel`. Diferentemente dos capítulos anteriores, aqui são incluídos trechos reais do projeto, porém algumas partes estratégicas foram omitidas ou substituídas por comentários para que o aluno complete a implementação por conta própria. A proposta é que o estudante compreenda a lógica e utilize pesquisa complementar para preencher os detalhes faltantes.

15.1 Objetivos

Ao final desta atividade, o kernel deverá:

- configurar interrupções periódicas de timer;
- tratar traps em modo supervisor;
- realizar troca de contexto automática;
- alternar entre tasks usando Round-Robin.

15.2 Arquivos Envolvidos

- `include/timer.h`
- `kernel/timer.c`
- `include/trap.h`
- `kernel/trap.c`
- `boot/trap_entry.S`
- `include/scheduler.h`
- `kernel/scheduler.c`
- `kernel/main.c`

15.3 Passo 1: Interface do Timer

15.3.1 timer.h

```
#ifndef TIMER_H
#define TIMER_H

#include <stdint.h>

void timer_init(uint64_t interval);
void timer_next(void);

#endif
```

15.3.2 Objetivo

- `timer_init()`: configura o timer periódico.
- `timer_next()`: agenda a próxima interrupção.

15.4 Passo 2: Implementação do Timer

15.4.1 timer.c

```
#include "timer.h"

static uint64_t tick_interval = 100000;

/* TODO:
 * Implementar chamada SBI set_timer.
 *
 * Protótipo sugerido:
 * static inline void sbi_set_timer(uint64_t time);
 */

void timer_next(void)
{
    uint64_t now;

    /* TODO:
     * Ler o CSR time.
     */

    /* TODO:
     * Programar now + tick_interval.
     */
}

void timer_init(uint64_t interval)
{
    if (interval != 0)
        tick_interval = interval;

    timer_next();

    /* TODO:
     * Habilitar STIE no CSR sie.
     */
}
```

```

    * Habilitar SIE global em sstatus.
    */
}

```

15.4.2 Desafio do Aluno

Pesquisar:

- chamada `ecall` para `set_timer`;
- bits STIE e SIE;
- leitura do CSR `time`.

15.5 Passo 3: Interface do Trap Handler

15.5.1 trap.h

```

#ifndef TRAP_H
#define TRAP_H

#include <stdint.h>

void trap_handler(uint64_t *frame);

#endif

```

15.6 Passo 4: Implementação do Trap Handler

15.6.1 trap.c

```

#include "trap.h"
#include "timer.h"
#include "scheduler.h"
#include "uart.h"

void trap_handler(uint64_t *frame)
{
    uint64_t scause;

    /* TODO:
     * Ler CSR scause.
     */

    /* TODO:
     * Verificar se interrupção.
     */

    /* TODO:
     * Identificar Supervisor Timer Interrupt.
     */

    /* TODO:
     * Chamar timer_next().
     * Chamar schedule_from_trap(frame).
     */
}

```

```

    uart_print("Unhandled trap\n");

    while (1)
        ;
}

```

15.6.2 Desafio do Aluno

Pesquisar:

- formato do CSR `scause`;
- código da interrupção de timer em S-mode.

15.7 Passo 5: Ponto de Entrada do Trap

15.7.1 `trap_entry.S`

```

.section .text
.global trap_entry

trap_entry:
    addi sp, sp, -256

    # TODO:
    # Salvar x1..x31 na stack.

    mv a0, sp
    call trap_handler

    # TODO:
    # Restaurar x1..x31 da stack.

    addi sp, sp, 256
    sret

```

15.7.2 Desafio do Aluno

Pesquisar:

- instruções `sd` e `ld`;
- convenção ABI do RISC-V;
- instrução `sret`.

15.8 Passo 6: Extensão da Interface do Scheduler

15.8.1 `scheduler.h`

```

#ifndef SCHEDULER_H
#define SCHEDULER_H

#include <stdint.h>

typedef int (*sched_algo_t)(void);

```

```

void scheduler_set_algorithm(sched_algo_t algo);
void scheduler_start(void);
void yield(void);

void schedule_from_trap(uint64_t *frame);

int scheduler_current_task(void);

#endif

```

15.9 Passo 7: Implementação do Round-Robin

15.9.1 Função Básica

```

static int round_robin(void)
{
    return (current + 1) % task_count;
}

```

Essa função retorna o índice da próxima task em ordem circular.

15.10 Passo 8: Escalonamento a Partir do Trap

15.10.1 Trecho de scheduler.c

```

void schedule_from_trap(uint64_t *frame)
{
    int prev = current;
    int next = current_algo();

    /* TODO:
     * Copiar frame -> tasks[prev].regs
     */

    /* TODO:
     * Salvar sepc da task atual.
     */

    current = next;

    /* TODO:
     * Copiar tasks[next].regs -> frame
     */

    /* TODO:
     * Restaurar sepc da próxima task.
     */
}

```

15.10.2 Desafio do Aluno

Pesquisar:

- CSRs sepc;
- uso de loops para copiar vetores.

15.11 Passo 9: Configuração no main.c

```
#include "task.h"
#include "scheduler.h"
#include "memory.h"
#include "timer.h"
#include "trap.h"

extern void uart_print(const char*);
extern void trap_entry(void);

void kernel_main()
{
    memory_init();

    uart_print("\n=== Kernel ===\n");

    xTaskCreate(task1, 2048, 1);
    xTaskCreate(task2, 2048, 1);

    asm volatile("csrw stvec, %0" :: "r"(trap_entry));

    timer_init(100000);

    scheduler_start();

    while (1);
}
```

15.12 Passo 10: Teste

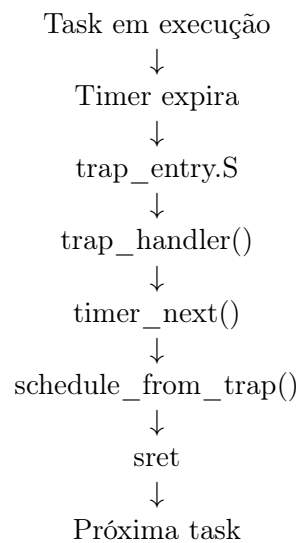
As tasks podem ser implementadas sem `yield()`:

```
void task1(void)
{
    while (1)
        uart_print("Task 1 running\n");
}

void task2(void)
{
    while (1)
        uart_print("Task 2 running\n");
}
```

Se a saída alternar automaticamente entre as tasks, o escalonamento preemptivo estará funcionando.

15.13 Fluxo Completo



15.14 Checklist

- Implementar `sbi_set_timer()`.
- Ler o CSR `time`.
- Habilitar bits em `sie` e `sstatus`.
- Ler o CSR `scause`.
- Identificar interrupção de timer.
- Salvar/restaurar registradores em `trap_entry.S`.
- Implementar `schedule_from_trap()`.
- Configurar `stvec`.

15.15 Resultado Esperado

Ao final, o kernel deverá exibir:

```
=== Kernel ===
Task 1 running
Task 2 running
Task 1 running
Task 2 running
...
```

sem que as tasks precisem chamar `yield()` explicitamente.

Capítulo 16

Proposta de Evolução do Sistema de Gerência de Memória

16.1 Introdução

Até este ponto do projeto, o microkernel utiliza um mecanismo extremamente simples de alocação dinâmica de memória, baseado em um modelo linear conhecido como:

Bump Allocator

Nesse modelo, o heap é tratado como uma região contínua de memória e cada nova alocação apenas avança um ponteiro indicando o topo atual do heap. Embora essa abordagem seja suficiente para demonstrar os conceitos iniciais de alocação dinâmica, ela possui limitações importantes que impedem a evolução natural do microkernel para funcionalidades mais próximas das presentes em sistemas operacionais reais. Por esse motivo, este capítulo propõe uma nova estratégia de gerenciamento de memória baseada em:

Free List Allocator com política First Fit

Essa implementação adiciona conceitos fundamentais utilizados em sistemas operacionais modernos, tais como:

- reutilização de memória;
- gerenciamento de blocos livres;
- fragmentação;
- desalocação dinâmica;
- divisão de blocos;
- coalescência de regiões adjacentes.

16.2 Limitações do Allocator Atual

A implementação atual do kernel utiliza um modelo semelhante ao seguinte:

```
void *kmalloc(uint64_t size)
{
    void *ptr = &heap[heap_top];

    heap_top += size;

    return ptr;
}
```

```
}
```

Esse modelo funciona da seguinte forma:

1. o heap é um vetor contínuo de memória;
2. `heap_top` aponta para o próximo byte livre;
3. cada chamada de `kmalloc()` reserva espaço sequencialmente;
4. o ponteiro avança continuamente.

16.2.1 Exemplo

Considere um heap inicialmente vazio:

```
+-----+
|                                             |
+-----+
^
heap_top
```

Após:

```
kmalloc(256);
```

temos:

```
+-----+-----+
| bloco alocado | livre |
+-----+-----+
^
heap_top
```

Após uma nova alocação:

```
kmalloc(128);
```

```
+-----+-----+-----+
| bloco 256 | bloco128 | livre |
+-----+-----+-----+
^
heap_top
```

16.3 Problemas do Modelo Linear

Apesar da simplicidade, o modelo possui diversas limitações importantes.

16.3.1 Ausência de `kfree()`

A memória nunca retorna ao heap. Uma vez alocada, ela permanece ocupada até o reinício do kernel. Isso impede:

- destruição dinâmica de tasks;
- reutilização de stacks;
- gerenciamento eficiente de recursos.

16.3.2 Exaustão Progressiva do Heap

Mesmo que uma task deixe de existir, sua memória continua ocupada. Consequentemente:

$$heap_top \rightarrow HEAP_SIZE$$

eventualmente causando falha em novas alocações.

16.3.3 Ausência de Reutilização

Imagine:

```
ptr1 = kmalloc(1024);  
ptr2 = kmalloc(1024);
```

Mesmo que `ptr1` não seja mais utilizado, não existe mecanismo para reaproveitar sua região.

16.3.4 Pouco Realismo

Sistemas operacionais reais implementam mecanismos sofisticados de gerenciamento de memória. O alocador linear é útil didaticamente apenas nas etapas iniciais.

16.4 Proposta: Free List Allocator

A proposta deste capítulo consiste em implementar um allocator baseado em:

Lista Encadeada de Blocos Livres

Nesse modelo:

- o heap é dividido em blocos;
- cada bloco possui um cabeçalho;
- blocos livres são reutilizados;
- blocos podem ser divididos;
- blocos adjacentes podem ser unidos.

16.5 Estrutura Conceitual do Heap

O heap deixa de ser apenas um vetor linear e passa a conter estruturas administrativas. Cada bloco possuirá:

Header
size
free
next
Dados do usuário

16.6 Estrutura do Cabeçalho

16.6.1 Arquivo `memory.c`

```
typedef struct block
{
    uint64_t size;

    int free;

    struct block *next;
} block_t;
```

16.6.2 Campo size

```
uint64_t size;
```

Armazena o tamanho útil do bloco, ou seja:

- não inclui o cabeçalho;
- representa apenas a área disponível ao usuário.

16.6.3 Campo free

```
int free;
```

Indica se o bloco está:

- livre;
- ocupado.

Valores possíveis:

- 0 = ocupado;
- 1 = livre.

16.6.4 Campo next

```
struct block *next;
```

Ponteiro para o próximo bloco do heap. Os blocos passam a formar uma lista encadeada.

16.7 Estrutura Inicial do Heap

Após:

```
memory_init();
```

todo o heap será tratado como um único bloco livre.

```
+-----+
| bloco livre único                               |
| tamanho = HEAP_SIZE - sizeof(block_t)          |
+-----+
```

16.8 Estratégia de Busca: First Fit

A política escolhida é:

First Fit

Nesse modelo:

1. percorre-se a lista de blocos;
2. o primeiro bloco livre suficientemente grande é utilizado.

16.9 Por que First Fit?

Essa estratégia foi escolhida porque:

- é simples de implementar;
- possui baixo custo computacional;
- é adequada para sistemas embarcados;
- possui excelente valor didático.

16.10 Comparação com Outras Estratégias

Estratégia	Vantagem	Desvantagem
First Fit	simples e rápida	fragmentação moderada
Best Fit	menor desperdício	busca mais lenta
Worst Fit	evita blocos pequenos	fragmentação futura

16.11 Novo Arquivo memory.h

```
#ifndef MEMORY_H
#define MEMORY_H

#include <stdint.h>

void memory_init(void);

void *kmalloc(uint64_t size);

void kfree(void *ptr);

uint64_t memory_used(void);

uint64_t memory_free(void);

#endif
```

16.12 Nova Organização do Heap

```
#define HEAP_SIZE 0x10000

static uint8_t heap[HEAP_SIZE];
```

```
static block_t *free_list;
```

16.12.1 Heap

```
static uint8_t heap[HEAP_SIZE];
```

Representa a memória total disponível para alocação dinâmica.

16.12.2 Lista Livre

```
static block_t *free_list;
```

Ponteiro para o primeiro bloco do heap.

16.13 Inicialização do Heap

16.13.1 Esqueleto

```
void memory_init(void)
{
    free_list = (block_t*) heap;

    free_list->size =
        HEAP_SIZE - sizeof(block_t);

    free_list->free = 1;

    free_list->next = 0;
}
```

16.13.2 Funcionamento

A inicialização:

1. converte o início do heap em um bloco;
2. define o tamanho total disponível;
3. marca o bloco como livre;
4. define o fim da lista.

16.14 Fluxo do kmalloc()

1. alinhar tamanho solicitado;
2. percorrer lista;
3. encontrar bloco livre;
4. dividir bloco;
5. marcar como ocupado;
6. retornar ponteiro.

16.15 Esqueleto do `kmalloc()`

```
void *kmalloc(uint64_t size)
{
    block_t *current = free_list;

    /* TODO:
     * Alinhar para mltiplos de 8 bytes.
     */

    while (current)
    {
        /* TODO:
         * Verificar:
         * - bloco livre
         * - tamanho suficiente
         */

        /* TODO:
         * Dividir bloco caso sobre espao.
         */

        /* TODO:
         * Marcar como ocupado.
         */

        /* TODO:
         * Retornar ponteiro aps cabealho.
         */

        current = current->next;
    }

    return 0;
}
```

16.16 Divisão de Blocos

Suponha:

```
+-----+
| bloco livre 1024 bytes |
+-----+
```

Após:

```
kmalloc(256);
```

o allocator pode gerar:

```
+-----+-----+
| usado256 | livre restante |
+-----+-----+
```

Esse processo é chamado:

Splitting

16.17 Fluxo do kfree()

1. localizar cabeçalho;
2. marcar bloco como livre;
3. unir blocos adjacentes.

16.18 Esqueleto do kfree()

```
void kfree(void *ptr)
{
    if (!ptr)
        return;

    /* TODO:
     * Recuperar cabeçalho do bloco.
     */

    /* TODO:
     * Marcar como livre.
     */

    /* TODO:
     * Implementar coalescência.
     */
}
```

16.19 Coalescência

Considere:

```
+-----+-----+
| livre | livre |
+-----+-----+
```

Esses blocos podem ser unidos:

```
+-----+
| bloco maior |
+-----+
```

Isso reduz:

Fragmentação Externa

16.20 Fragmentação

16.20.1 Fragmentação Interna

Ocorre quando o bloco é maior que o necessário.

16.20.2 Fragmentação Externa

Ocorre quando existem vários blocos livres pequenos espalhados.

16.21 Esqueleto do memory_used()

```
uint64_t memory_used(void)
{
    block_t *current = free_list;

    uint64_t used = 0;

    while (current)
    {
        /* TODO:
         * Somar blocos ocupados.
         */

        current = current->next;
    }

    return used;
}
```

16.22 Esqueleto do memory_free()

```
uint64_t memory_free(void)
{
    block_t *current = free_list;

    uint64_t free_mem = 0;

    while (current)
    {
        /* TODO:
         * Somar blocos livres.
         */

        current = current->next;
    }

    return free_mem;
}
```

16.23 Integração com Tasks

Atualmente:

```
t->stack = kmalloc(stack_size);
```

Com o novo allocator:

- stacks poderão ser reutilizadas;
- tasks poderão ser destruídas;
- memória poderá retornar ao heap.

16.24 Atividades Propostas

O aluno deverá:

1. implementar alinhamento de memória;
2. implementar busca First Fit;
3. implementar divisão de blocos;
4. implementar `kfree()`;
5. implementar coalescência;
6. implementar estatísticas do heap.

16.25 Pesquisas Recomendadas

Pesquisar:

- buddy allocator;
- slab allocator;
- SLUB do Linux;
- allocators embarcados;
- fragmentação em sistemas operacionais.

16.26 Saída Esperada

O kernel poderá exibir:

```
Heap total: 65536 bytes
Heap usado: 4096 bytes
Heap livre: 61440 bytes
Fragmentacao: 12%
```

16.27 Objetivos de Aprendizagem

Ao concluir esta atividade, o aluno deverá ser capaz de:

- implementar um allocator baseado em lista livre;
- compreender fragmentação;
- implementar desalocação dinâmica;
- reutilizar memória;
- comparar diferentes estratégias de alocação.

16.28 Conclusão

A substituição do allocator linear por um Free List Allocator representa uma evolução significativa do microkernel. Essa abordagem aproxima o projeto de sistemas operacionais reais, permitindo estudar conceitos fundamentais utilizados em kernels modernos e fornecendo uma base sólida para futuras extensões do sistema.

Capítulo 17

Implementação de um Sistema de Arquivos Hierárquico Baseado em Inodes (TreeFS)

17.1 Introdução

Os sistemas de arquivos desempenham um papel fundamental em qualquer sistema operacional, sendo responsáveis por organizar, armazenar e recuperar informações de forma persistente. À medida que a capacidade dos dispositivos de armazenamento aumentou e a quantidade de dados manipulados pelos computadores cresceu, tornou-se necessário desenvolver mecanismos capazes de organizar grandes volumes de informações de maneira eficiente e escalável. Considere um computador moderno utilizado por múltiplos usuários. Nesse ambiente podem coexistir simultaneamente:

- arquivos do sistema operacional;
- bibliotecas compartilhadas;
- arquivos temporários;
- documentos pessoais;
- projetos de desenvolvimento;
- imagens;
- vídeos;
- arquivos de configuração;
- bancos de dados.

Se todos esses arquivos fossem armazenados em um único local, a organização rapidamente se tornaria inviável. A localização de informações específicas seria cada vez mais difícil, além de aumentar significativamente a complexidade de gerenciamento do sistema. Para resolver esse problema, os sistemas operacionais modernos adotaram uma abordagem hierárquica para a organização dos arquivos. Nessa abordagem, arquivos e diretórios são organizados em uma estrutura de árvore, permitindo a criação de múltiplos níveis de diretórios e subdiretórios.

Essa organização facilita a separação lógica das informações, melhora a navegabilidade do sistema e permite que milhões de arquivos sejam armazenados de forma estruturada. Além disso, essa abordagem torna possível a implementação de mecanismos avançados de gerenciamento, segurança e compartilhamento de dados. Os sistemas Unix foram pioneiros na utilização dessa

arquitetura, introduzindo conceitos que permanecem presentes até hoje em praticamente todos os sistemas operacionais modernos, incluindo Linux, BSD, Android e macOS. Entre esses conceitos destaca-se o uso dos **inodes**, estruturas responsáveis por armazenar os metadados associados aos arquivos e diretórios.

Neste capítulo será proposta a implementação de um sistema de arquivos denominado **TreeFS**, inspirado nos princípios encontrados em sistemas como Minix File System, EXT2 e XV6. O objetivo não é reproduzir integralmente esses sistemas, mas compreender os principais mecanismos utilizados por sistemas de arquivos hierárquicos modernos por meio de uma implementação simplificada e adequada ao contexto educacional do BasicMicrokernel.

17.2 Objetivos de Aprendizagem

Ao concluir esta atividade, o aluno deverá ser capaz de:

- compreender a organização hierárquica de sistemas de arquivos;
- compreender o conceito de inode;
- compreender a diferença entre metadados e dados;
- implementar gerenciamento de blocos;
- implementar gerenciamento de inodes;
- implementar resolução de caminhos;
- implementar diretórios hierárquicos;
- compreender a arquitetura dos sistemas de arquivos Unix.

17.3 Sistemas de Arquivos Hierárquicos

Os primeiros sistemas computacionais armazenavam quantidades relativamente pequenas de arquivos. Em muitos casos, os arquivos eram mantidos em estruturas simples, sem a necessidade de múltiplos níveis de organização. Considere a seguinte estrutura:

```
/
notas.txt
trabalho.pdf
foto.jpg
video.mp4
```

Enquanto o número de arquivos permanece reduzido, essa organização é suficiente. Entretanto, em ambientes reais, a quantidade de informações cresce rapidamente. Um sistema operacional moderno precisa armazenar arquivos pertencentes ao próprio sistema, aplicações instaladas e dados de diversos usuários simultaneamente. Uma organização mais realista poderia ser representada por:

```
/home/aluno/projetos
/home/aluno/documentos
/home/aluno/imagens
/home/professor
/etc
/bin
/usr
```

Nesse cenário, torna-se evidente a necessidade de uma estrutura capaz de organizar informações em múltiplos níveis hierárquicos. A utilização de diretórios e subdiretórios oferece

diversas vantagens:

- melhor organização das informações;
- maior facilidade de navegação;
- separação lógica dos dados;
- melhor escalabilidade;
- redução da complexidade de gerenciamento.

Para suportar esse modelo de organização, os sistemas Unix introduziram uma arquitetura baseada em diretórios hierárquicos e inodes, permitindo que arquivos e diretórios sejam organizados em estruturas semelhantes a árvores. Essa abordagem tornou-se a base da maioria dos sistemas de arquivos modernos e será utilizada como fundamento para o desenvolvimento do TreeFS ao longo deste capítulo.

17.4 Por Que Sistemas Hierárquicos Existem?

Imagine uma biblioteca contendo milhares de livros. Se todos os livros fossem colocados em uma única estante, localizar um título específico seria extremamente difícil. Uma solução natural consiste em dividir os livros em categorias:

- Ciência;
- Matemática;
- História;
- Literatura;
- Engenharia.

Dentro de cada categoria podem existir subdivisões. Por exemplo:

```
Engenharia
|
+-- Elétrica
|
+-- Mecânica
|
+-- Computação
```

Os sistemas de arquivos modernos seguem exatamente essa lógica. Em vez de armazenar todos os arquivos em um único local, eles são organizados em diretórios e subdiretórios. Essa organização facilita:

- localização de arquivos;
- gerenciamento do sistema;
- separação entre usuários;
- escalabilidade.

17.5 Entendendo Estruturas Hierárquicas

Uma árvore é uma estrutura de dados composta por nós conectados por relações de pai e filho. O nó localizado no topo da árvore é denominado raiz. Nos sistemas Unix, a raiz é representada por:

/

Todos os demais elementos do sistema de arquivos são descendentes dessa raiz. Exemplo:

/

```
home
  aluno
    notas.txt
    projeto.c

  professor

bin
  shell
  editor
```

```
etc
  config
```

Observe que:

- diretórios podem conter arquivos;
- diretórios podem conter outros diretórios;
- não existe limite conceitual para a profundidade da árvore.

Essa característica torna possível organizar milhões de arquivos de forma estruturada.

17.6 A Analogia com uma Árvore Genealógica

Uma forma intuitiva de compreender a estrutura hierárquica consiste em compará-la com uma árvore genealógica. Considere:

```
Avô
|
+-- Pai
|   |
|   +-- Filho 1
|   |
|   +-- Filho 2
|
+-- Tio
    |
    +-- Primo
```

Cada elemento possui um pai e pode possuir múltiplos filhos. Da mesma forma:

```
/
|
+-- home
|   |
|   +-- aluno
|       |
|       +-- notas.txt
|
+-- etc
```


|
+-- bin

Nesse exemplo:

- "/" é pai de home;
- home é pai de aluno;
- aluno é pai de notas.txt.

Essa relação forma a estrutura hierárquica do sistema.

17.7 Metadados e Dados

Antes de estudar os inodes é necessário compreender a diferença entre metadados e dados. Os dados representam o conteúdo efetivo de um arquivo. Exemplo:

Sistemas Operacionais

Já os metadados representam informações sobre esse conteúdo. Exemplos de metadados:

- tamanho do arquivo;
- tipo;
- localização dos blocos;
- proprietário;
- permissões;
- timestamps.

Uma analogia simples consiste em imaginar um livro. O texto do livro corresponde aos dados. A ficha catalográfica corresponde aos metadados.

17.8 O Que é um Inode?

O inode é uma das estruturas mais importantes dos sistemas Unix. O termo inode significa:

Index Node

Um erro muito comum entre alunos iniciantes consiste em acreditar que o inode contém o nome do arquivo. Na realidade isso não acontece. O inode não conhece o nome do arquivo. O inode contém apenas os metadados. O nome é armazenado separadamente nos diretórios. Essa separação é uma das decisões de projeto mais elegantes dos sistemas Unix.

17.9 A Analogia com uma Biblioteca

Uma analogia extremamente útil consiste em comparar o sistema de arquivos a uma biblioteca. Nesse cenário:

- o livro representa os dados;
- a ficha catalográfica representa o inode;
- o catálogo representa os diretórios.

Visualmente:

Catálogo

```

    |
    v
Nome do Livro
    |
    v
Ficha Catalográfica
    |
    v
Localização Física

```

Observe que a ficha não contém o livro. Ela apenas informa onde ele está. O mesmo ocorre com os inodes.

17.10 Inodes em Detalhes

O inode funciona como uma ficha cadastral associada a um arquivo. Entre as informações normalmente armazenadas em um inode encontram-se:

- tamanho;
- tipo;
- quantidade de referências;
- localização dos blocos de dados.

Uma estrutura simplificada para o TreeFS é apresentada abaixo.

```

typedef struct{
    uint32_t size;
    uint16_t type;
    uint16_t links;
    uint32_t blocks[8];
} inode_t;

```

Cada arquivo e cada diretório existente no sistema possuirá exatamente um inode associado.

17.11 Campos do Inode

Após compreender a estrutura geral do inode, é importante analisar detalhadamente a função de cada um de seus campos. Cada campo foi projetado para armazenar uma informação específica necessária para o gerenciamento dos arquivos e diretórios do sistema.

17.11.1 Campo Size

O campo:

```
uint32_t size;
```

armazena o tamanho do arquivo em bytes.

Por exemplo, considere o arquivo:

notas.txt

contendo:

Sistemas Operacionais

Esse texto possui 22 bytes. Portanto:

```
inode->size = 22;
```

O campo size permite que o sistema saiba exatamente quantos bytes devem ser lidos durante uma operação de leitura. Sem essa informação, seria impossível distinguir os dados válidos do espaço não utilizado existente no último bloco do arquivo.

17.11.2 Campo Type

O campo:

```
uint16_t type;
```

indica qual o tipo do objeto representado pelo inode. Uma definição típica seria:

```
#define FS_FILE 1
#define FS_DIR 2
```

Exemplos:

```
inode->type = FS_FILE;
```

ou

```
inode->type = FS_DIR;
```

Essa informação é extremamente importante porque o kernel precisa tratar arquivos e diretórios de maneira diferente. Por exemplo:

- arquivos podem ser lidos e escritos;
- diretórios podem conter outros arquivos e diretórios.

17.11.3 Campo Links

O campo:

```
uint16_t links;
```

armazena a quantidade de referências existentes para o inode. Embora inicialmente esse campo possa parecer desnecessário, ele desempenha um papel fundamental nos sistemas Unix. Considere:

arquivo.txt

Se apenas uma entrada apontar para o inode:

```
inode->links = 1;
```

o sistema sabe que existe apenas uma referência para aquele arquivo. Quando múltiplos nomes apontam para o mesmo inode, o campo links permite controlar quantas referências ainda existem. O arquivo somente será removido quando:

```
links == 0
```

17.11.4 Campo Blocks

O campo:

```
uint32_t blocks[8];
```

armazena os blocos físicos utilizados pelo arquivo.

Considere:

arquivo.txt

ocupando:

Bloco 10

Bloco 15

Bloco 20

O inode armazenaria:

```
blocks[0] = 10;  
blocks[1] = 15;  
blocks[2] = 20;
```

Observe uma diferença importante em relação ao FAT. No FAT:

Cluster -> FAT -> Próximo Cluster

No TreeFS:

Inode -> Lista de Blocos

O próprio inode contém a lista de blocos utilizados.

17.12 Arquitetura Geral do TreeFS

A arquitetura proposta para o TreeFS é apresentada abaixo.

```
+-----+  
| Aplicações / Tasks      |  
+-----+  
| API de Arquivos        |  
+-----+  
| TreeFS                  |  
+-----+  
| Driver de Blocos       |  
+-----+  
| Disco Virtual          |  
+-----+
```

Cada camada possui uma responsabilidade específica. As aplicações realizam operações como leitura e escrita. O TreeFS gerencia diretórios, inodes e blocos. O driver de blocos realiza o acesso ao dispositivo de armazenamento.

17.13 Organização Física do Disco

Assim como ocorreu no SimpleFAT, o TreeFS organiza o dispositivo de armazenamento em regiões especializadas.

```
+-----+
```

Superblock	
+-----+	
Bitmap de Inodes	
+-----+	
Bitmap de Blocos	
+-----+	
Área de Inodes	
+-----+	
Área de Dados	
+-----+	

Essa organização é inspirada nos sistemas Minix e EXT2. Cada região possui uma função específica.

17.14 Superblock

O Superblock é a estrutura mais importante do sistema de arquivos. Ele funciona como uma espécie de documento de identidade do TreeFS. Quando o sistema operacional monta um sistema de arquivos, a primeira estrutura lida normalmente é o Superblock. Uma estrutura simplificada pode ser:

```
typedef struct{
    uint32_t magic;
    uint32_t total_blocks;
    uint32_t total_inodes;
    uint32_t block_size;
} superbblock_t;
```

O campo magic funciona como uma assinatura. Por exemplo:

```
#define TREEFS_MAGIC 0x54524653
```

Ao montar o sistema:

```
if(sb.magic != TREEFS_MAGIC)
{
    erro();
}
```

o kernel consegue verificar se aquele dispositivo realmente contém um TreeFS válido.

17.15 Gerenciamento de Inodes Livres

O sistema precisa saber quais inodes estão livres. Para isso será utilizado um bitmap. Considere:

Índice : 0 1 2 3 4 5 6 7

Valor : 1 1 0 0 1 0 0 0

Convenção:

- 0 = livre;
- 1 = ocupado.

Nesse exemplo:

inode 2

é o primeiro inode disponível.

17.16 Bitmap de Inodes

Uma implementação simplificada pode utilizar:

```
uint8_t inode_bitmap[MAX_INODES];
```

Fluxo para alocação:

```
Percorrer bitmap
|
Encontrar posição livre
|
Marcar como ocupada
|
Retornar inode
```

17.17 Bitmap de Blocos

O gerenciamento dos blocos livres segue exatamente a mesma lógica.

```
uint8_t block_bitmap[MAX_BLOCKS];
```

Convenção:

- 0 = bloco livre;
- 1 = bloco ocupado.

17.18 Primeira Estrutura do Sistema

Uma dificuldade comum durante a implementação de sistemas de arquivos hierárquicos consiste em determinar como iniciar a árvore. Após a formatação do TreeFS, será criada automaticamente uma estrutura mínima.

```
/
home
tmp
bin
```

Essa estrutura permitirá validar a navegação antes mesmo da implementação completa da criação dinâmica de diretórios. Por exemplo:

```
ls("/");
```

Saída esperada:

```
home
tmp
bin
```

17.19 Inicialização do Sistema

A função responsável pela inicialização será:

```
int fs_init(void);
```

Durante a inicialização deverão ser realizadas as seguintes operações:

1. inicializar o Superblock;
2. limpar os bitmaps;
3. criar o inode raiz;
4. criar os diretórios iniciais;
5. preparar a estrutura de armazenamento.

Fluxo:

```
fs_init()
|
Inicializar Superblock
|
Inicializar Bitmaps
|
Criar Inode Raiz
|
Criar /home
|
Criar /tmp
|
Criar /bin
|
Sistema Pronto
```

17.20 Alocação de Inodes

A função responsável pela alocação será:

```
inode_t *inode_alloc(void);
```

Seu objetivo consiste em localizar um inode livre. Passos:

1. percorrer o bitmap;
2. localizar uma posição livre;
3. marcar como ocupada;
4. retornar o inode correspondente.

A complexidade dessa implementação é:

$$O(n)$$

onde n representa a quantidade de inodes disponíveis.

17.21 Liberação de Inodes

Quando um arquivo é removido, seu inode deve retornar ao conjunto de recursos livres.

Fluxo:

```
Receber inode
|
Localizar índice
|
Marcar bitmap como livre
|
Limpar inode
```

Isso permitirá que o inode seja reutilizado futuramente.

17.22 Diretórios

Uma das características mais elegantes dos sistemas Unix consiste em tratar diretórios como arquivos especiais. Do ponto de vista do kernel, um diretório é armazenado da mesma forma que um arquivo comum. A principal diferença está no conteúdo armazenado. Enquanto um arquivo comum contém dados do usuário:

Relatório da disciplina

um diretório contém referências para outros arquivos e diretórios. Essas referências associam nomes aos respectivos inodes. Visualmente:

```
home
|
+--> aluno      inode 10
|
+--> professor  inode 12
|
+--> visitante  inode 20
```

Essa estrutura permite que o sistema localize rapidamente qualquer arquivo da árvore.

17.23 Entradas de Diretório

Cada diretório é composto por diversas entradas. Uma entrada associa um nome a um inode. Uma possível estrutura para representar uma entrada de diretório é:

```
typedef struct{
    uint32_t inode;
    char name[32];
} dir_entry_t;
```

Observe que o diretório não armazena os dados do arquivo. Ele apenas informa:

- o nome;
- o inode associado.

Por exemplo:

```
notas.txt  -> inode 15
projeto.c  -> inode 21
foto.jpg   -> inode 30
```

Quando o usuário solicita a abertura de um arquivo, o sistema primeiro localiza o inode correspondente e somente depois acessa os dados.

17.24 Resolução de Caminhos

A resolução de caminhos é uma das funcionalidades mais importantes do TreeFS. Seu objetivo consiste em transformar uma string contendo um caminho em um inode. Considere:

`/home/aluno/notas.txt`

Para localizar esse arquivo, o sistema precisa percorrer toda a árvore. Visualmente:

```
/
|
+-- home
   |
   +-- aluno
       |
       +-- notas.txt
```

O processo ocorre em etapas. Primeiro:

`/`

Depois:

`home`

Em seguida:

`aluno`

Finalmente:

`notas.txt`

Somente após percorrer todos os níveis da árvore o inode correto poderá ser retornado. Esse processo é conhecido como travessia da árvore.

17.25 Implementação de `path_lookup()`

A função responsável pela resolução de caminhos será:

```
inode_t *
path_lookup(const char *path);
```

Entrada:

`/home/aluno/notas.txt`

Saída:

`inode correspondente`

Fluxo:

Receber caminho

|

Iniciar na raiz

|

Separar componentes

|

Localizar próximo diretório

|

Entrar no diretório

```
    |
Repetir processo
    |
Retornar inode final
```

Essa função será utilizada por praticamente todas as operações do sistema de arquivos.

17.26 Navegação na Árvore

Uma vez implementada a resolução de caminhos, torna-se possível navegar pela árvore. Uma função simples para validação pode ser:

```
int ls(const char *path);
```

Exemplo:

```
ls("/");
```

Saída:

```
home
tmp
bin
```

Outro exemplo:

```
ls("/home");
```

Saída:

```
aluno
professor
```

Essa funcionalidade é extremamente útil durante os testes do sistema.

17.27 Criação de Diretórios

A criação de diretórios é uma das operações fundamentais do TreeFS. Exemplo:

```
mkdir("/home/aluno");
```

Internamente, o sistema precisa realizar diversas operações. Primeiramente deve localizar o diretório pai. Em seguida deve verificar se já existe uma entrada com o mesmo nome. Depois disso um novo inode deve ser alocado. Por fim, uma nova entrada deve ser adicionada ao diretório pai. Fluxo:

```
Receber caminho
    |
Localizar pai
    |
Verificar duplicidade
    |
Alocar inode
    |
Inicializar diretório
    |
Adicionar entrada
```

17.28 Implementação de mkdir()

Protótipo:

```
int mkdir(const char *path);
```

Passos recomendados:

1. localizar o diretório pai;
2. verificar se o nome já existe;
3. alocar inode;
4. inicializar metadados;
5. criar entrada de diretório;
6. atualizar estruturas de controle.

Exemplo:

Antes:

```
/
home
```

Depois:

```
/
home
  aluno
```

17.29 Criação de Arquivos

A criação de arquivos segue uma lógica semelhante à criação de diretórios. Exemplo:

```
create("/home/aluno/notas.txt");
```

Fluxo:

```
Receber caminho
|
Localizar diretório pai
|
Verificar duplicidade
|
Alocar inode
|
Inicializar arquivo
|
Adicionar entrada
```

Resultado:

```
/home/aluno
|
+--> notas.txt
```

17.30 Implementação de create()

Protótipo:

```
int create(const char *path);
```

Objetivos:

- criar inode;
- criar entrada de diretório;
- inicializar tamanho;
- preparar estrutura para escrita futura.

Inicialmente o arquivo poderá possuir tamanho zero.

17.31 Escrita de Arquivos

Após a criação do arquivo, o próximo passo consiste em permitir o armazenamento de dados. Considere:

```
write(fd,  
      buffer,  
      size);
```

A operação de escrita deverá:

1. localizar o inode;
2. determinar quantos blocos serão necessários;
3. alocar blocos livres;
4. copiar os dados;
5. atualizar o tamanho do arquivo.

Visualmente:

```
Arquivo  
  |  
Inode  
  |  
Blocos  
  |  
Dados
```

17.32 Implementação de write()

Protótipo:

```
int write(int fd,  
          const void *buf,  
          uint32_t size);
```

Fluxo:

```
Receber buffer  
  |
```

Localizar inode
|
Calcular blocos
|
Alocar blocos
|
Copiar dados
|
Atualizar inode

Ao final da operação, o campo:

<code>inode->size</code>

deverá refletir o tamanho real do arquivo.

17.33 Leitura de Arquivos

A leitura realiza o processo inverso da escrita. Seu objetivo consiste em reconstruir o conteúdo armazenado a partir das informações presentes no inode. Fluxo:

Arquivo
|
Inode
|
Lista de Blocos
|
Dados
|
Buffer

O sistema deverá percorrer os blocos associados ao inode e copiar seu conteúdo para o buffer fornecido pela aplicação.

17.34 Implementação de read()

Protótipo:

<code>int read(int fd, void *buf, uint32_t size);</code>
--

Passos:

1. localizar inode;
2. identificar blocos utilizados;
3. ler os blocos;
4. reconstruir os dados;
5. copiar para o buffer.

17.35 Remoção de Arquivos

Quando um arquivo deixa de ser necessário, seus recursos devem retornar ao sistema. A remoção envolve mais do que simplesmente apagar o nome do arquivo. É necessário:

- remover a entrada do diretório;
- liberar blocos;
- liberar inode;
- atualizar bitmaps.

Fluxo:

```
Localizar arquivo
|
Remover entrada
|
Liberar blocos
|
Liberar inode
|
Atualizar bitmaps
```

17.36 Implementação de unlink()

Protótipo:

```
int unlink(const char *path);
```

Passos:

1. localizar inode;
2. liberar blocos utilizados;
3. atualizar bitmap de blocos;
4. atualizar bitmap de inodes;
5. remover entrada do diretório.

Após a conclusão da operação, os recursos anteriormente utilizados pelo arquivo poderão ser reutilizados por novos arquivos.

17.37 Validação da Implementação

Uma vez concluída a implementação, o sistema deverá ser capaz de executar operações semelhantes às encontradas em sistemas Unix. Exemplo:

```
mkdir("/home");
mkdir("/home/aluno");
create("/home/aluno/notas.txt");
ls("/home/aluno");
```

Saída esperada:

```
notas.txt
```

Posteriormente:

```
write(fd, "Sistemas Operacionais", 22);
```

e:

```
read(fd, buffer, 22);
```

deverão permitir armazenar e recuperar informações corretamente.

17.38 Conclusão

O TreeFS introduz conceitos fundamentais encontrados em praticamente todos os sistemas de arquivos modernos baseados em Unix. Ao longo deste capítulo foram apresentados os mecanismos responsáveis pelo gerenciamento de diretórios hierárquicos, inodes, blocos de armazenamento e resolução de caminhos. Embora simplificado, o TreeFS preserva os princípios fundamentais utilizados por sistemas reais, servindo como base para futuras evoluções, incluindo suporte a permissões, timestamps, links simbólicos, blocos indiretos e sistemas de arquivos mais sofisticados. A compreensão desses conceitos representa um passo fundamental na formação de profissionais capazes de projetar e implementar sistemas operacionais completos.